Highlights

Fault tolerant control of a redundant cable driven parallel robot with model based fault isolation and precomputed recovery plans

Mfeuter Joseph Tachia, Alexander V. Maloletov

- A 17 mode IMM isolates every cable break within 19 ms from winch signals alone
- Recovery plans computed offline keep the platform after 197 of 208 cable breaks
- A learned classifier in the same loop declared later and raised more false alarms
- Probability weighted MPC gave no benefit, as breaks are declared within one period
- A one period delay of the MPC output made the platform fall before any fault

Fault tolerant control of a redundant cable driven parallel robot with model based fault isolation and precomputed recovery plans

Mfeuter Joseph Tachia[a,*], Alexander V. Maloletov[a,b]

[a]Institute of Robotics and Computer Vision, Innopolis University, Universitetskaya St. 1, Innopolis, 420500, Russia
[b]Volgograd State Technical University, Lenin Avenue 28, Volgograd, Russia

ARTICLE INFO

Keywords:
Cable driven parallel robot
Fault tolerant control
Fault isolation
Interacting multiple model
Model predictive control
Cable failure
Recovery planning

ABSTRACT

A cable driven parallel robot that loses a cable has a fraction of a second to move its platform to a pose that the remaining cables can still hold. We present a fault tolerant controller for the eight cable robot IPAnema 2 that identifies the fault from the motor encoders and currents of the winches alone and answers a cable break with a recovery motion computed in advance. Extended state observers estimate the cable tensions at each winch, and an interacting multiple model estimator over seventeen modes separates cable breaks from actuator faults at the platform. A model predictive controller tracks the task within the tension bounds and, once a break is declared, follows a stored optimal recovery path that brings the platform to rest at a pose the seven remaining cables can hold. The controller was compared with eight alternatives in a paired simulation study of 624 fault cases and 200 fault free runs on randomised robots with elastic cables, winch dynamics and sensor noise. It identified 623 of the 624 faults correctly, declared every cable break within 19 ms and lost the platform after 11 of 208 cable breaks, against 122 without fault tolerance and 65 with a classical threshold based pipeline. A recurrent neural classifier trained on the same signals identified faults later and less reliably and raised frequent false alarms, and when its probabilities also weighted the fault scenarios of the controller, the platform was lost in 44 of 64 break runs. Weighting the fault scenarios of the controller by their probabilities before a declaration, the idea this study set out to test, gave no measurable benefit, because the model based diagnosis declares a cable break within one control period. All results assume that each solution of the MPC acts within its sample: in an added study with a protocol of its own, an output delayed by one period of 20 ms made the platform fall before any fault in all 104 runs tested, so a real time implementation that meets this assumption is the main step before tests on hardware.

1. Introduction

A cable driven parallel robot can lose a cable in the middle of a task, and when it does, the platform starts to fall before any controller has noticed. These robots move a platform with cables wound on winches fixed to a frame, which gives them a workspace of several metres, a light moving mass and a payload that rigid parallel robots of the same size cannot reach [1–3]. These properties are the reason they are considered for building facades, large scale additive manufacturing and other construction tasks [4], and they are also the reason a failure is dangerous: a platform of several kilograms hangs metres above the floor, held by thin cables and by drives that run for hours. A cable can break, and a winch can lose part of its torque or develop a parasitic one. In a fully constrained robot with eight cables and six degrees of freedom the loss of one cable does not make the platform uncontrollable at once, but it removes most of the poses at which the remaining cables can hold it. For the robot of this study, IPAnema 2 [5], no level pose near the task can be held after an upper cable breaks (Section 2.6), and without a fault tolerant reaction the platform was lost after every one of 104 simulated breaks of an upper cable. Recovering from a break is therefore a race. The controller has to find out which cable has gone, decide where the platform can still be held and start moving it there within a few tenths of a second, while the platform is already falling.

Most work on cable failures asks what the robot can still do once the failure is known. Roberts et al. [6] studied the fault tolerance of cable suspended robots through their statics, and Notash [7] gave criteria under which the wrench capability of a wire actuated manipulator is fully or partly recovered after a failure. For the motion after a break, Berti et al. [8] moved the point mass of a redundant suspended robot from the unstable pose left by the failure to a new static equilibrium, Passarini et al. [9] drove a cable suspended camera toward a safe landing pose along an oscillatory trajectory that keeps the remaining tensions positive and bounded, and Boschetti et al. [10] stopped a rehabilitation robot with a wrench opposed to its motion. Boumann and Bruckmann [11] developed emergency strategies that minimise the kinetic energy of the platform or steer it with potential fields, and validated them later on a prototype [12], while Di Paola et al. [13] guided the load inside the remaining wrench feasible workspace with a sliding mode adaptive PID control. These strategies take the failure as known at its onset, and most of them were shown on planar or point mass models or on suspended robots. For a spatial, fully constrained robot two questions are left open: how the delay and the reliability of the diagnosis shape the outcome, and how a recovery motion of the full six degree of freedom pose, which has to be dynamically feasible with

*Corresponding author
m.tachia@innopolis.university (M.J. Tachia)
ORCID(s): 0000-0002-9274-3658 (M.J. Tachia); 0000-0002-7312-2944 (A.V. Maloletov)

seven cables, can be ready within the few tenths of a second the platform leaves.

Banks of mode matched filters combined by the interacting multiple model (IMM) estimator [14] have long been used to detect and isolate actuator and sensor failures [15, 16]. In cable driven robots the diagnosis has received less attention than the recovery. Raman et al. [17] isolated cable failures of a planar reconfigurable robot with an IMM bank of extended Kalman filters, assuming that the platform pose is measured. Majdi et al. [18] detected partial actuator losses of a planar robot with an extended state observer, and Fazeli et al. [19] estimated actuator faults with an adaptive observer inside a finite time sliding mode controller and verified the scheme on a planar robot. Bettega et al. [20] estimated the load torque of each motor from its current and encoder and detected cable failures with machine learning classifiers. Learned detectors need no model of the faulty robot, but they know only the behaviour represented in their training data [21], and in a fault tolerant controller that behaviour changes as soon as the controller acts on the detector. None of these works isolates both cable breaks and actuator faults of a spatial robot from the winch signals alone, and we have not found a comparison of a learned and a model based diagnosis inside the same closed loop.

Model predictive control (MPC) suits cable robots because the tension bounds become constraints of the optimisation instead of being left to a separate tension distribution step [22]. Santos et al. [23] built the tension distribution into a nonlinear MPC with conditions for stability, Song and Lau [24] used an offline workspace analysis to give the online problem convex constraints and recursive feasibility, and Gao et al. [25] combined MPC with a disturbance observer. When the model itself is uncertain, multistage MPC optimises one input sequence against several candidate models at once [26]. A diagnosis that returns mode probabilities suggests combining the two: before a fault is confirmed, the controller could already weigh what each plausible fault would do and move the platform toward poses that would survive it. This was the hypothesis our study began with, and here it is tested as an ablation.

This paper presents and evaluates a fault tolerant controller for IPAnema 2 on a simulator that integrates elastic and unilateral cables, the winch dynamics, sensor quantisation and noise and randomised parameters, while the controller works with a rigid cable model. An extended state observer at each winch estimates the cable tension from the encoder and the motor current, and an IMM over seventeen modes, the healthy robot, eight cable breaks and eight actuator faults, separates breaks from actuator faults at the platform. An MPC tracks the task within the tension bounds, and after a break is declared it follows a recovery plan, the solution of an optimal control problem on the seven cable model computed in advance along the known task. All methods ran on the same randomised runs, with the endpoints, the statistics and the settings of every method fixed before the evaluation and one documented amendment of the threshold baseline (Section 5.1). The contributions are three.

1. A two level diagnosis that isolates cable breaks, losses of effectiveness and torque biases of a spatial robot from the encoders and motor currents of the winches, without load cells or a pose sensor. It identified 623 of 624 faults correctly and declared every break within 19 ms.
2. A library of recovery plans computed offline along the task and selected at the declaration, which removes the latency of solving the plan online. With it the platform was lost after 11 of 208 breaks, against 20 with the same planner solved online and 122 without fault tolerance.
3. Paired evidence on when weighting the fault scenarios of the MPC by their probabilities before a declaration can act. With the IMM it changed no safety endpoint, because every break was declared within one period of the MPC. With a recurrent classifier that shares the state estimator of the IMM and was trained on the same signals, the weighting came with false alarms in 154 of 252 runs and with unsuccessful solves of the MPC, and the platform was lost in 44 of 64 break runs.

We report the result on the weighting as a finding and not as a defect of the design: hedging before a declaration can only act while the diagnosis stays uncertain for several control periods, and with the one such diagnosis we tested it did not help. We do not claim new theory for the observers, the estimator or the MPC; the contribution is their combination for a spatial robot that has neither load cells nor a pose sensor, and the paired evidence on what it achieves. All results are simulations in which each computation is available at its sample, and Section 6.8 shows that the controller depends on this: with the output of the MPC one period late, the platform fell before any fault.

The argument runs from the robot to the evaluation. Section 2 derives the model of IPAnema 2 and shows what a cable break leaves of its workspace, which fixes what the controller must do. Sections 3 and 4 build the diagnosis and the controller on that model, Section 5 describes how they were evaluated against eight alternatives, Section 6 gives the results, and Section 7 discusses what they show and where the study stops.

2. Robot, model and faults

2.1. The robot

IPAnema 2 is a member of a family of industrial cable robots, built in a frame of $8 \times 6 \times 5$ m [5]. We use its geometry, mass, inertia and upper tension bound as distributed with the IPAnema 2 model of the open source toolkit CASPR [27], whose geometry coincides with the published one, and its cables, winches and drives as published (Table 1). Anchor i is at $\boldsymbol{a}_i = (4\sigma_{x,i}, 3\sigma_{y,i}, z_a)$ m, with $z_a = 5$ m for the upper cables 1 to 4 and $z_a = 1$ m

Table 1
Model of the robot. The controller uses the nominal values; the plant of every run draws the randomised ones from the seed of the run.

Quantity	Value	Source
Frame (anchor box)	8×6 m, anchors at $z = 1$ and 5 m	CASPR model IPAnema_2 [27]
Platform attachments	$\lvert x\rvert \le 0.75$, $\lvert y\rvert \le 0.125$, $z \in [0.25, 0.75]$ m	CASPR
Mass, inertia	10 kg, 1 kg m^2 per axis	CASPR
Tension bounds	$t_{\min} = 10$ N, $t_{\max} = 200$ N	CASPR ($t_{\max}$); design choice ($t_{\min}$)
Cable	Dyneema, $d = 1.5$ mm, $EA = 35188$ N, retardation 20 ms	[5]; EA scaled from [28]
Drum, gearbox	radius 50 mm, $i = 12$, stroke 6 m	[5]
Motor	MSK030C: 0.9 N m rated, 4 N m peak, rotor 3×10^{-5} kg m^2	sized from [29]
Velocity, acceleration limits	2.5 m/s, 25 m/s^2	[5]
Encoder	1048576 counts/rev, ± 120 arcsec	[29], option M1
Motor current noise	0.004 N m (nominal), $\times 2$ in half of the runs	design choice
Rates	plant RK4 step 62.5 μs, control 1000 Hz, measurement delay 1 sample	step convergence study
Randomised in every run	mass $\pm 10\%$, principal inertias $\pm 20\%$, centre of mass ± 2 cm, anchors ± 5 mm, EA $\pm 20\%$ (uniform)	study design

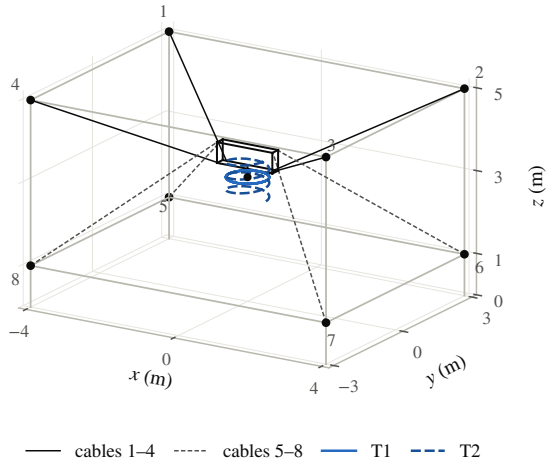


Figure 1: IPAnema 2 at its home pose, with the eight cables (upper anchors at $z = 5$ m, lower anchors at $z = 1$ m), the reference point P of the platform and the two reference paths: the circle T1 and the helix T2, both 16 s long.

for the lower cables 5 to 8, and the cable is attached to the platform at $\boldsymbol{b}_i = (0.65\sigma_{x,i}, 0.125\sigma_{y,i}, 0.25)$ m for the upper and $(0.75\sigma_{x,i}, 0.10\sigma_{y,i}, 0.75)$ m for the lower cables, where the signs $(\sigma_{x,i}, \sigma_{y,i})$ are $(-,+)$, $(+,+)$, $(+,-)$ and $(-,-)$ for cables 1 to 4 and, in the same order, for cables 5 to 8. The platform of 10 kg has its centre of mass 0.5 m above its reference point P, the point that follows the task, and the home pose places P at $(0,0,3)$ m with zero orientation (Fig. 1). With eight cables pulling upward and downward at the same time the platform resists a wrench in any direction while every cable stays in tension. The redundancy is two, which leaves a two dimensional family of tension distributions for every wrench, and after the loss of one cable it drops to one.

2.2. Kinematics

The world frame has its z axis pointing upward. The pose of the platform is described by the generalised coordinates

$$\boldsymbol{q} = \begin{bmatrix} \boldsymbol{p} \\ \boldsymbol{\varphi} \end{bmatrix} \in \mathbb{R}^6, \qquad \mathbf{R}(\boldsymbol{\varphi}) = \mathbf{R}_z(\psi)\,\mathbf{R}_y(\vartheta)\,\mathbf{R}_x(\phi), \quad (1)$$

where $\boldsymbol{p}$ is the position of P and $\boldsymbol{\varphi} = [\phi, \vartheta, \psi]^\top$ are ZYX Euler angles. Each cable is taken to leave the frame at its fixed anchor point, and the small motion of the exit point on the guiding pulley is neglected. The angular velocity follows from the Euler rates as $\boldsymbol{\omega} = \mathbf{E}(\boldsymbol{\varphi})\dot{\boldsymbol{\varphi}}$, with $\mathbf{E}$ singular only at $\vartheta = \pm\pi/2$, far outside the tilt this robot can hold, and we collect the velocities in the twist $\boldsymbol{\nu} = [\dot{\boldsymbol{p}}^\top, \boldsymbol{\omega}^\top]^\top = \mathbf{H}(\boldsymbol{q})\dot{\boldsymbol{q}}$ with $\mathbf{H} = \mathrm{blkdiag}(\mathbf{I}_3, \mathbf{E})$. Cable i runs from the attachment point to the anchor, so its vector, length and direction are

$$\boldsymbol{l}_i = \boldsymbol{a}_i - \boldsymbol{p} - \mathbf{R}\boldsymbol{b}_i, \qquad l_i = \|\boldsymbol{l}_i\|, \qquad \boldsymbol{u}_i = \boldsymbol{l}_i / l_i, \quad (2)$$

and a tension $t_i \ge 0$ acts on the platform as the force $t_i\boldsymbol{u}_i$. Differentiating (2) gives, for all cables together,

$$\dot{\boldsymbol{l}} = -\mathbf{W}^\top \boldsymbol{\nu}, \qquad \mathbf{W} = \begin{bmatrix} \boldsymbol{u}_1 & \cdots & \boldsymbol{u}_n \\ (\mathbf{R}\boldsymbol{b}_1) \times \boldsymbol{u}_1 & \cdots & (\mathbf{R}\boldsymbol{b}_n) \times \boldsymbol{u}_n \end{bmatrix}, \quad (3)$$

where $\mathbf{W} \in \mathbb{R}^{6\times n}$ is the wrench matrix: the cables apply to the platform the force and the moment about P given by $\mathbf{W}\boldsymbol{t}$. The same matrix, transposed and with a minus sign, maps the twist to the rates of the cable lengths, and the sign in (3) is the one that keeps the cables pulling.

2.3. Dynamics from the Lagrange equations

We treat the platform as a rigid body and the cables, in this subsection, as sources of generalised forces. With $\boldsymbol{c}_w = \mathbf{R}\boldsymbol{c}$ the position of the centre of mass relative to P, the kinetic and potential energies are

$$T = \tfrac{1}{2}m\|\dot{\boldsymbol{p}} + \boldsymbol{\omega} \times \boldsymbol{c}_w\|^2 + \tfrac{1}{2}\boldsymbol{\omega}^\top \mathbf{I}_c \boldsymbol{\omega} = \tfrac{1}{2}\dot{\boldsymbol{q}}^\top \mathbf{M}(\boldsymbol{q})\dot{\boldsymbol{q}}, \quad (4)$$

$$V = mg\,\boldsymbol{e}_3^{\top}(\boldsymbol{p} + \mathbf{R}\boldsymbol{c}), \tag{5}$$

where $\mathbf{I}_c = \mathbf{R}\mathbf{I}_G\mathbf{R}^{\top}$ is the inertia about the centre of mass in world coordinates, and expanding the square gives

$$\mathbf{M} = \mathbf{H}^{\top}\mathbf{M}_v\mathbf{H}, \qquad \mathbf{M}_v = \begin{bmatrix} m\mathbf{I}_3 & -m[\boldsymbol{c}_w]_{\times} \\ m[\boldsymbol{c}_w]_{\times} & \mathbf{I}_c - m[\boldsymbol{c}_w]_{\times}^2 \end{bmatrix}, \tag{6}$$

with $[\cdot]_{\times}$ the cross product matrix. The off diagonal blocks couple translation and rotation because P is not the centre of mass, and with an offset of 0.5 m this coupling is far from small. The elastic energy of the cables is deliberately left out of V: the cable forces enter once, through their virtual work, so that they cannot be counted both as a potential and as an applied force. A virtual displacement moves attachment point i by $\delta\boldsymbol{p} + \delta\boldsymbol{\Theta} \times \mathbf{R}\boldsymbol{b}_i$ with $\delta\boldsymbol{\Theta} = \mathbf{E}\,\delta\boldsymbol{\varphi}$, and the virtual work of the tensions,

$$\delta W = \sum_{i=1}^{n} t_i \boldsymbol{u}_i^{\top}(\delta\boldsymbol{p} + \delta\boldsymbol{\Theta} \times \mathbf{R}\boldsymbol{b}_i) = (\mathbf{H}^{\top}\mathbf{W}\boldsymbol{t})^{\top}\delta\boldsymbol{q}, \tag{7}$$

gives the generalised force $\boldsymbol{Q} = \mathbf{H}^{\top}\mathbf{W}\boldsymbol{t} = -(\partial\boldsymbol{l}/\partial\boldsymbol{q})^{\top}\boldsymbol{t}$, which says that a tension does negative work when its cable gets longer. The Lagrange equations of $L = T - V$ then read

$$\mathbf{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \mathbf{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{G}(\boldsymbol{q}) = \mathbf{H}^{\top}\mathbf{W}\boldsymbol{t}, \tag{8}$$

where $\boldsymbol{G} = \partial V/\partial\boldsymbol{q} = -\mathbf{H}^{\top}\boldsymbol{w}_g$ with the gravity wrench $\boldsymbol{w}_g = [m\boldsymbol{g}^{\top},\ (\boldsymbol{c}_w \times m\boldsymbol{g})^{\top}]^{\top}$, and $\mathbf{C}$ is built from the Christoffel symbols of the first kind,

$$C_{kj} = \sum_{i=1}^{6} \frac{1}{2}\Big(\frac{\partial M_{kj}}{\partial q_i} + \frac{\partial M_{ki}}{\partial q_j} - \frac{\partial M_{ij}}{\partial q_k}\Big)\dot{q}_i, \tag{9}$$

the choice that makes $\dot{\mathbf{M}} - 2\mathbf{C}$ skew symmetric. Substituting $\dot{\boldsymbol{q}} = \mathbf{H}^{-1}\boldsymbol{\nu}$ and multiplying by $\mathbf{H}^{-\top}$ gives the Newton–Euler form about P,

$$\begin{aligned} \mathbf{M}_v\dot{\boldsymbol{\nu}} + \boldsymbol{b}(\boldsymbol{q}, \boldsymbol{\nu}) &= \mathbf{W}\boldsymbol{t} + \boldsymbol{w}_g, \\ \boldsymbol{b} &= \begin{bmatrix} m\,\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{c}_w) \\ \boldsymbol{\omega} \times \mathbf{I}_c\boldsymbol{\omega} + m\,\boldsymbol{c}_w \times \big(\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{c}_w)\big) \end{bmatrix}, \end{aligned} \tag{10}$$

which has no Euler angle singularity in its mass matrix and is cheaper to evaluate. The simulator integrates (10) with the orientation carried as a unit quaternion, so that it stays regular through the large rotations of an uncontrolled platform, and the estimators and the controller use the same equation with the tensions as inputs. We checked the derivation against a symbolic Lagrangian and the two forms against each other numerically (Appendix A).

2.4. Cables and winches

Each cable is wound on a drum that a motor turns through a gearbox. With the motor angle θ_i taken positive when cable is reeled in, the unstretched free length is $l_{u,i} = l_{u,i}^0 - r_d\theta_i$, where $r_d = r_D/i_g = 50\ \text{mm}/12 = 4.167$ mm/rad is the cable paid out per radian of the motor shaft. With the stretch $\Delta_i = l_i - l_{u,i}$ and the axial stiffness $k_i = EA/l_{u,i}$, the tension follows a Kelvin–Voigt law cut off at zero,

$$t_i = \begin{cases} \max\big(0,\ k_i\Delta_i + \beta k_i\, s(\Delta_i)\,\dot{\Delta}_i\big), & \Delta_i > 0, \\ 0, & \Delta_i \le 0, \end{cases} \tag{11}$$

where β is a retardation time. The cut off is what makes a cable a cable: it pulls but cannot push, and once slack it carries no load until it is stretched again. The factor $s(\Delta) = 3r^2 - 2r^3$, $r = \min(\Delta/\Delta_s, 1)$ with $\Delta_s = 0.1$ mm, lets the damping grow from zero with the stretch, as Hunt and Crossley [30] proposed for impacts, so that a cable which tightens again does not jump at once to the force $\beta k\dot{\Delta}$. No stiffness has been published for the 1.5 mm Dyneema cables of IPAnema 2. We scaled the values identified on the 6 mm cable of IPAnema 3 [28] by the ratio of the cross sections, which gives $EA = 35.2$ kN, kept the retardation time of 0.02 s, and randomised EA by $\pm 20\%$ in every run because the value is scaled and not measured. At the home pose the six elastic modes of the platform with locked winches then lie between 5.5 and 18.7 Hz. The cables are taken as straight: over the feasible workspace the equivalent modulus of a sagging cable [31] never falls below 95.4% of EA. This is a property of the light cable of this robot. With heavier cables the sag changes the cable forces noticeably, and models that combine the Kelvin–Voigt elasticity with a catenary have been used to compensate it with a learned controller [32].

Every winch carries a synchronous servo motor of the Bosch Rexroth IndraDyn S series with a multi turn absolute encoder, a planetary gearbox with $i_g = 12$ and a drum of 100 mm diameter that stores 6 m of cable [5]. The motor size of IPAnema 2 is not published, and we took the MSK030C, the smallest motor of the series that reaches the speed and the holding torque the robot needs [29]. Referred to the motor shaft, the winch obeys

$$I_m\ddot{\theta}_i = \tau_i^{\text{act}} - r_d t_i, \tag{12}$$

where τ_i^{act} is the torque the motor delivers and I_m the rotor inertia; friction is left out, as in the winch analysis of Idà and Mattioni [33], because no data on it are published. The winch is not small next to the platform: seen from the cable, the rotor of 0.3 kg cm^2 acts like a mass of $I_m/r_d^2 = 1.7$ kg, against a platform of 10 kg. Equation (12) is the Lagrange equation of the coordinate θ_i, with the kinetic energy $\frac{1}{2}I_m\dot{\theta}_i^2$ and the virtual work $(\tau_i^{\text{act}} - r_d t_i)\,\delta\theta_i$ of the motor and the cable on the drum, so that (8) and (12) together are the Lagrange equations of the robot in the coordinates $(\boldsymbol{q}, \boldsymbol{\theta})$. The drum limits the change of the free length to 6 m, a window that we centred on the lengths each cable takes over the static workspace. Equations (10), (11) and (12) form the model of the simulator.

2.5. Faults and sensors

Three faults are described by one parameterisation,

$$\tau_i^{\text{act}} = (1 - \rho_i)\,\tau_i + f_i, \qquad t_i \equiv 0 \text{ for } t \ge t_f \text{ if cable } i \text{ breaks}, \tag{13}$$

where τ_i is the torque the drive applies. A break removes the cable for good, a loss of effectiveness $\rho_i \in (0,1)$ models a drive that delivers only part of its torque, for instance through a damaged gear stage, and a bias f_i models a constant parasitic torque such as a dragging brake.

The robot carries no load cells and no sensor of the platform pose. Each winch reports its motor angle, from the encoder, and its motor current, which the torque constant turns into the torque τ_i; on the IPAnema robots the same current is used to estimate the cable tension [5]. This choice is deliberate. With a load cell on every cable a break would show as a zero reading and the diagnosis would be trivial, whereas without load cells the tension has to be inferred from the motor side, and a loss of effectiveness becomes invisible to the drive, whose current is correct while its torque is not. The encoder resolves 2^{20} counts per revolution with an error of $\pm 120''$ [29], and the torque measurement carries white noise of 0.004 N m, a value we chose because none is published and which the evaluation doubles in half of the runs. The controller runs at 1 kHz and receives every reading one sample late.

2.6. What a cable break leaves

For a set $\mathscr{S}$ of intact cables, a pose is statically feasible when every free length lies in its drum window and the cables of $\mathscr{S}$ balance gravity with tensions inside $[t_{\min}, t_{\max}]$. How far inside that set a pose lies is measured by the margin

$$s^\star(\boldsymbol{q},\mathscr{S}) = \max_{s,\boldsymbol{t}_\mathscr{S}} s \quad \text{s.t.} \quad \begin{aligned} &\mathbf{W}_\mathscr{S}\boldsymbol{t}_\mathscr{S} + \boldsymbol{w}_g = \mathbf{0}, \\ &t_{\min} + s \le t_i \le t_{\max} - s, \end{aligned} \tag{14}$$

a linear programme whose solution is also the tension distribution farthest from both bounds. On a grid of 0.25 m over the interior of the frame, the eight cables hold the level platform at 3,357 positions within the drum windows. After the loss of one upper cable 11.8% of these positions remain, and after the loss of a lower cable 42.2%. Fig. 2 shows two slices through the paths. In the horizontal plane $z = 3$ m, where the circular task lies, 8,059 of 14,241 points of a 0.05 m grid are feasible with all cables, 14 after the loss of the upper cable 1 and 4,073 after the loss of the lower cable 5.

The robot keeps more of its workspace when the platform may give up its orientation. Allowing a tilt of up to 20° about each axis, 52.3% of the positions can be held after an upper cable breaks and 73.8% after a lower one, with a median tilt of 10°. Tilting is therefore the main resource the robot has left after a break. Redistributing the tensions and carrying on is not an option, since for most of the workspace there is no distribution to carry on with; the platform has to be moved and tilted into the feasible set of the cables that remain, and the motion has to start before that set is out of reach.

The two reference paths of the evaluation lie in the centre of the workspace (Fig. 1). T1 is a horizontal circle of 0.5 m radius at $z = 3$ m with a period of 4 s, followed for three turns in 16 s, and the helix T2 follows the same circle while climbing from $z = 2.5$ to 3.5 m. The peak speed of 0.79 m/s is a third of the speed limit of the robot. On neither path can the level platform be held at any point after an upper cable breaks, while after a lower cable breaks it can on 29 to 52% of the path, depending on the cable. An upper cable break therefore always forces the platform to leave its orientation, and a lower cable break sometimes leaves the task possible at a lower speed.

3. Fault diagnosis

The diagnosis has to tell eight cable breaks and eight actuator faults apart from the encoders and motor currents, and it has to do it in a few tens of milliseconds. It works on two levels because the two kinds of fault leave their signatures in different places.

3.1. Tension estimation at each winch

Writing (12) with (13) as $\ddot{\theta}_i = b_0\tau_i + d_i$ with $b_0 = 1/I_m$ gathers everything that is not the commanded torque into one unknown, $d_i = -(r_d t_i + \rho_i\tau_i - f_i)/I_m$. An extended state observer [34] treats d_i as a third state and estimates it from the encoder,

$$\begin{aligned} \dot{z}_{1,i} &= z_{2,i} + 3\omega_o(\tilde{\theta}_i - z_{1,i}), \\ \dot{z}_{2,i} &= z_{3,i} + b_0\tau_i + 3\omega_o^2(\tilde{\theta}_i - z_{1,i}), \\ \dot{z}_{3,i} &= \omega_o^3(\tilde{\theta}_i - z_{1,i}), \end{aligned} \tag{15}$$

with all observer poles at $-\omega_o$ [35], and the tension estimate is $\hat{t}_i = -I_m z_{3,i}/r_d$. For a bounded rate of d_i the error of such an observer is ultimately bounded, with a bound that shrinks as ω_o grows [36], but ω_o cannot grow freely, because the third channel amplifies the encoder quantisation by ω_o^3. With $\omega_o = 2\pi \cdot 50$ rad/s the estimate followed the cable tension of the plant with an RMS error of 0.30 N at the nominal current noise and 0.57 N at twice that noise, at the price of a lag of $3/\omega_o = 9.5$ ms. Comparing d_i with $z_{3,i}$, the estimate converges to

$$\hat{t}_i \;\to\; t_i + \frac{\rho_i\tau_i - f_i}{r_d}. \tag{16}$$

This relation is the reason the diagnosis needs two levels. A cable break is seen at the winch, because t_i, and with it $\hat{t}_i$, drops to zero while the commanded tension does not. An actuator fault is not seen there, because it adds to $\hat{t}_i$ exactly the torque the motor failed to deliver: a tension loop closed on $\hat{t}_i$ holds the estimate at its reference while the true tension is lower by the deficit $\delta_i = (\rho_i\tau_i - f_i)/r_d$. The deficit acts on the platform, and the platform is where it has to be found. Each winch tracks the tension $t_{d,i}$ requested by the controller with

$$\tau_i = r_d t_{d,i} + I_m\ddot{\theta}_i^{\mathrm{ff}} + r_d\Big(k_P e_i + k_I\int e_i\,dt\Big), \qquad e_i = t_{d,i} - \hat{t}_i, \tag{17}$$

where $\ddot{\theta}_i^{\mathrm{ff}}$ is the drum acceleration implied by the motion the controller predicts.

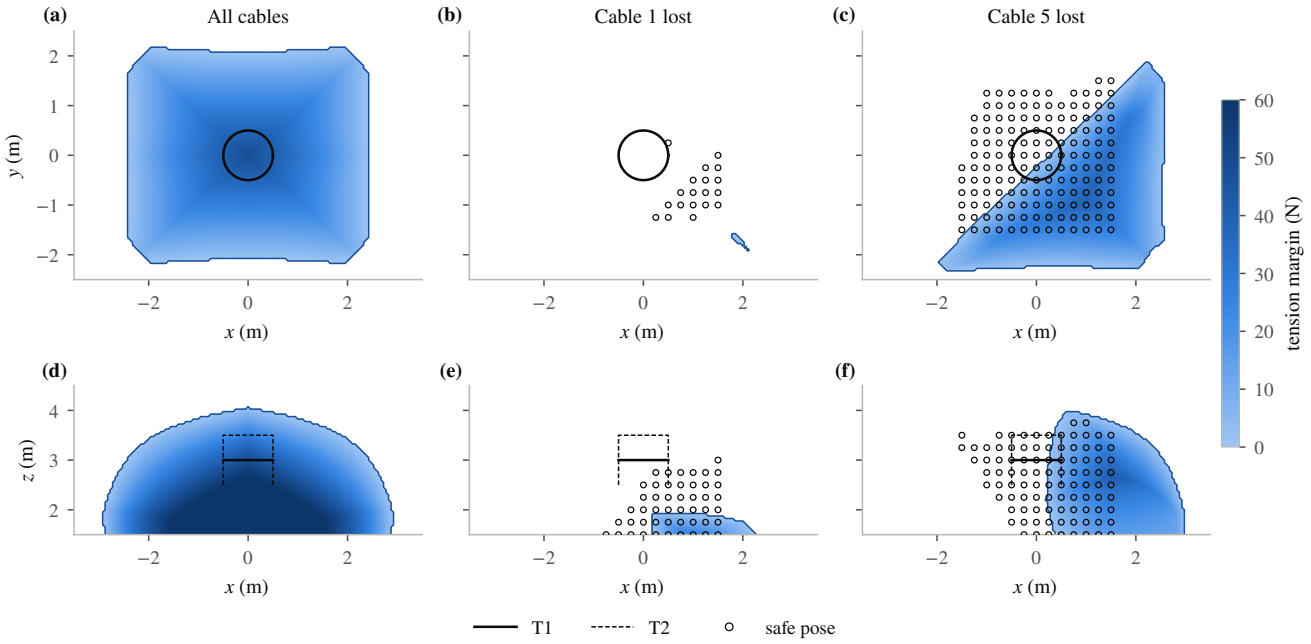


Figure 2: A cable break leaves the level platform almost no pose near the task when the lost cable is an upper one. Shading: tension margin (14) of the level platform at the poses where all remaining cables stay within $[t_{\min}, t_{\max}] = [10, 200]$ N and within their drum windows, in the horizontal plane $z = 3$ m (a–c) and the vertical plane $y = 0$ (d–f), with all cables (a, d), after the loss of the upper cable 1 (b, e) and after the loss of the lower cable 5 (c, f). Circles: poses in the same plane that the remaining cables hold with a margin of at least 10 N once the platform may tilt by up to 20°; these are the targets of the recovery plans. Lines: the path T1 and the extent of the helix T2.

3.2. Interacting multiple model estimator

The platform level diagnosis runs an IMM estimator [14] over the $N = 2n + 1 = 17$ modes $\mathscr{M}_0$ (healthy), $\mathscr{B}_i$ (break of cable i) and $\mathscr{A}_i$ (actuator fault of winch i). Each mode states which tensions the platform receives for a commanded vector $\boldsymbol{t}_d$,

$$\boldsymbol{t}^{(0)} = \boldsymbol{t}_d, \qquad \boldsymbol{t}^{(\mathscr{B}_i)} = (\mathbf{I} - \boldsymbol{e}_i \boldsymbol{e}_i^{\top})\boldsymbol{t}_d, \qquad \boldsymbol{t}^{(\mathscr{A}_i)} = \boldsymbol{t}_d - \boldsymbol{e}_i \delta_i, \tag{18}$$

where δ_i is the unknown deficit of (16). The mode $\mathscr{A}_i$ merges the loss of effectiveness and the bias, because both produce the same signature at the platform. Every filter carries the state $\boldsymbol{x} = [\boldsymbol{q}^{\top}, \boldsymbol{v}^{\top}, \delta, \boldsymbol{w}_b^{\top}, \boldsymbol{\eta}^{\top}, \gamma]^{\top} \in \mathbb{R}^{28}$. The deficit δ is a random walk used only by the modes $\mathscr{A}_i$. The other three groups describe what the nominal model gets wrong and are shared by all modes: $\boldsymbol{w}_b$ is a slowly varying wrench that absorbs the errors in mass, centre of mass and inertia, $\boldsymbol{\eta}$ holds offsets of the length readings that absorb the errors of the anchors and of the homing, and γ is the ratio of the true to the nominal axial stiffness. They are needed because the deficit of a mode $\mathscr{A}_i$ is a free parameter: without them, on a randomised plant, the filters of the actuator modes absorbed the model mismatch and won in fault free runs. A fault, in contrast, is an abrupt change that the slow bias states cannot follow on the time scale of the diagnosis, so it remains for the fault modes to explain.

Each filter propagates the mean of (10), with its own $\boldsymbol{t}^{(j)}$ and the wrench $\boldsymbol{w}_b$ added, over the period $\Delta_I = 5$ ms by a fourth order Runge–Kutta step, and the covariance with the Jacobian obtained by algorithmic differentiation. The measurement is $\boldsymbol{y} = [\tilde{\boldsymbol{l}}_u^{\top}, \hat{\boldsymbol{t}}^{\top}]^{\top}$, the unstretched lengths read from the encoders and the tension estimates of the winches, and under mode j its expected value is

$$\tilde{l}_{u,i} = \frac{\gamma EA\, l_i(\boldsymbol{q})}{\gamma EA + t_i^{(j)}} + \eta_i, \qquad \hat{t}_i = \begin{cases} 0, & j = \mathscr{B}_i, \\ t_{d,i}, & \text{otherwise}, \end{cases} \tag{19}$$

where the first relation is (11) solved for the free length and the second states that the tension loop holds $\hat{t}_i$ at its reference unless the cable has gone. The filters take the tension commands, averaged over the period, as their input rather than the tension estimates, which lag the true tensions by 9.5 ms; this also gives the IMM a model of the motion the controller itself produces. During the first 2 s the fault modes receive no prior probability, which gives the bias states time to converge at the start of the task. The predicted probabilities are bounded below by 10^{-300} to keep their logarithms finite, so a fault mode can still win in that interval, but only when the healthy filter misses the measurements by an extreme margin; this happened in no run of the evaluation and in most runs of the latency study of Section 6.8.

Let π_{lj} be the probability of switching from mode l to mode j in one period and μ_l the mode probabilities

Table 2
Settings of the estimators and the controller. Design: chosen and stated, not tuned; calibrated: chosen on calibration runs by rules written before those runs.

Part	Setting	Value	Source
Winch observer and loop	bandwidth $\omega_o/2\pi$; k_P, k_I	50 Hz; 0.5, 20 s^{-1}	design
IMM	period; arming time	5 ms; 2 s	design
	process noise, linear and angular acceleration	0.5 $\mathrm{m\,s}^{-2}$, 1 $\mathrm{rad\,s}^{-2}$	design; calibrated
	measurement noise, lengths and tensions	0.2 mm, 1.5 N	design
	deficit random walk; fault and return rates	20 $\mathrm{N\,s}^{-1/2}$; 10^{-3}, 10^{-2} s^{-1}	design
Declaration	μ_{iso}, N_c	0.9999, 2	calibrated
MPC	period, horizon	20 ms, 12 steps	design
	weights: pose; velocity; tension change	4×10^4; 120; 0.03	calibrated
	after a break declaration	velocity $\times0.3$, tension change $\times3$	calibrated
	scenarios: threshold μ_s, at most	10^{-3}, 2 fault scenarios	design
Planner	time scaling: horizon, $\kappa_{\min}$	2 s, 0.25	design
	recovery (23): T, intervals; s_p, $s_{\min}$	1 s, 15; 5, 10 N	design
	library: break instants; detection delay	every 50 ms; 15 ms	design

after the previous period. One cycle computes the predicted probabilities and the mixing weights,

$$\bar{c}_j=\sum_l \pi_{lj}\mu_l, \qquad \mu_{l|j}=\pi_{lj}\mu_l/\bar{c}_j, \tag{20}$$

mixes the states that all filters share, runs the extended Kalman prediction and update of each filter, and obtains its innovation $\boldsymbol{r}^{(j)}$ with covariance $\mathbf{S}^{(j)}$. The likelihood of mode j is the Gaussian density of its innovation, and the probabilities follow from Bayes' rule,

$$\ln\Lambda^{(j)}=-\tfrac{1}{2}\big(\boldsymbol{r}^{(j)\top}\mathbf{S}^{(j)-1}\boldsymbol{r}^{(j)}+\ln\det(2\pi\mathbf{S}^{(j)})\big), \qquad \mu_j=\frac{\Lambda^{(j)}\bar{c}_j}{\sum_l\Lambda^{(l)}\bar{c}_l}. \tag{21}$$

We evaluate (21) in the log domain, subtracting the largest $\ln\Lambda^{(l)}+\ln\bar{c}_l$ before exponentiating. This is not a detail. A cable break produces innovations of tens of newtons, the densities of all modes then underflow in double precision, and a direct evaluation returns $0/0$ or falls back on the prior, after which the probabilities only follow the Markov chain. In the log domain $\mu_j\geq0$ and $\sum_j\mu_j=1$ hold at every step, and the code asserts both. A fault is declared when one mode $j\neq0$ holds $\mu_j\geq\mu_{\mathrm{iso}}$ for N_c consecutive cycles. The noise levels, the switching rates and the pair $\mu_{\mathrm{iso}}=0.9999$, $N_c=2$ are listed in Table 2, with those chosen on calibration runs marked.

4. Control and recovery

Fig. 3 shows how the parts fit together, and Algorithm 1 gives the order in which they run.

4.1. Model predictive control with fault scenarios

At each control instant the scenario set $\mathscr{J}=\{0\}\cup\{j:\mu_j\geq\mu_s\}$ holds the healthy mode and the fault modes whose probability exceeds $\mu_s=10^{-3}$, at most the two most probable ones, with weights $\tilde{\mu}_j=\mu_j/\sum_{l\in\mathscr{J}}\mu_l$. The controller solves

$$\begin{aligned}\min_{\boldsymbol{t}_0,\dots,\boldsymbol{t}_{N-1}}\ &\sum_{j\in\mathscr{J}}\tilde{\mu}_j\Big[\sum_{k=0}^{N-1}\big(\|\boldsymbol{x}_k^{(j)}-\boldsymbol{x}_k^{\mathrm{ref}}\|^2_{\mathbf{Q}_x}+\|\boldsymbol{t}_k-\boldsymbol{t}_{k-1}\|^2_{\mathbf{Q}_t}\big)\\ &\qquad+V_f^{(j)}(\boldsymbol{x}_N^{(j)})\Big]\\ \text{s.t. }&\boldsymbol{x}_{k+1}^{(j)}=\boldsymbol{f}_d\big(\boldsymbol{x}_k^{(j)},\boldsymbol{t}^{(j)}(\boldsymbol{t}_k)\big),\quad \boldsymbol{x}_0^{(j)}=\hat{\boldsymbol{x}}^{(j)},\\ &t_{\min}\leq t_{k,i}\leq t_{\max},\qquad j\in\mathscr{J},\end{aligned} \tag{22}$$

where $\boldsymbol{x}_k^{(j)}=(\boldsymbol{q},\boldsymbol{v})$ is the platform state predicted under mode j, $\boldsymbol{f}_d$ is (10) discretised over the control period and $\boldsymbol{t}^{(j)}(\cdot)$ the map (18). All scenarios share one tension sequence, so the first command suits every mode that is

Algorithm 1 One cycle of the controller

Every 1 ms: read the encoder angles and motor torques of the previous sample
update the observer (15) of every winch; $\hat{t}_i\leftarrow-I_m z_{3,i}/r_d-\hat{\delta}_i$
if winch i is declared faulty
if 5 ms have passed since the last IMM cycle **then**
  run the IMM (20)–(21) with the averaged commands; obtain $\boldsymbol{\mu}$, $\hat{\boldsymbol{x}}$, $\hat{\delta}_i$
  if no fault is declared and $\mu_j\geq\mu_{\mathrm{iso}}$ in N_c cycles for one $j\neq0$ **then**
    declare j; the MPC predicts with mode j from now on
    if $j=\mathscr{B}_i$ **then**
      take the plan of cable i for the instant nearest $t-15$ ms; scale the MPC weights
if 20 ms have passed since the last MPC solve **then**
  solve (22) from $\hat{\boldsymbol{x}}$ (scenarios from $\boldsymbol{\mu}$ before a declaration); obtain $\boldsymbol{t}_d$, $\ddot{\boldsymbol{\theta}}^{\mathrm{ff}}$
apply the tension loop (17) of every winch

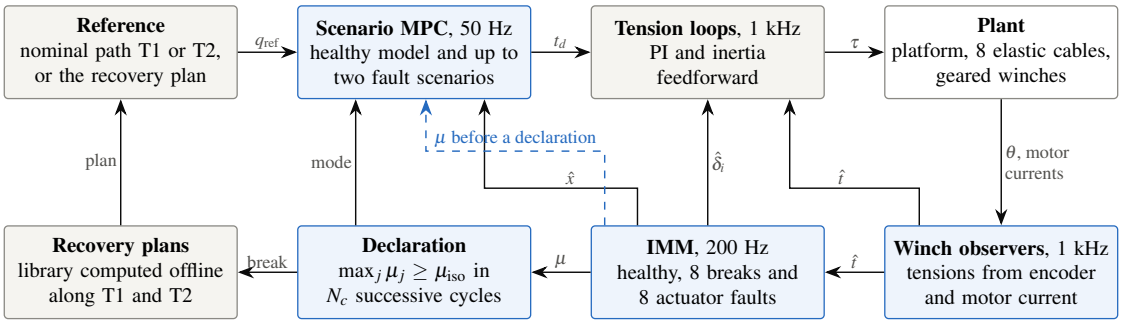


Figure 3: The fault tolerant controller. The winch observers estimate the tensions $\hat{\boldsymbol{t}}$ from the encoder angles and motor currents; the IMM, driven by the tension commands $\boldsymbol{t}_d$ and fed with $\hat{\boldsymbol{t}}$ and the encoder lengths, returns the mode probabilities $\boldsymbol{\mu}$, the platform state $\hat{\boldsymbol{x}}$ and, for an actuator fault, the deficit $\hat{\delta}_i$ that the tension loop of that winch compensates. Before a declaration the MPC can weight up to two fault scenarios by their probabilities (dashed); after a break declaration its reference becomes the recovery plan stored for that cable and instant.

still plausible. For the healthy mode the terminal cost $V_f^{(0)}$ penalises the distance to the reference; for a fault mode it pulls the predicted pose toward the nearest pose that the remaining cables hold with a margin of 10 N, looked up from sets stored offline. As μ_j grows, this term starts moving the platform toward the region in which it would survive fault j before the fault is confirmed; we call this the probability weighting of the scenarios. The probabilities weight costs only: the tension bounds are the same box for every scenario and do not depend on them. Raman et al. [17] used IMM weights to blend the outputs of mode specific controllers, whereas in (22) the probabilities weight predicted costs inside one optimisation, so the tension bounds hold whichever mode is true. With the healthy mode alone, (22) is a tracking MPC with tension bounds whose first command also solves the tension distribution problem. In the implementation the predictions also carry the bias wrench $\hat{\boldsymbol{w}}_b$ of the IMM, the stage cost of a fault scenario is scaled by 0.1, and two small terms are added, a pull toward the feedforward tensions of the reference and a penalty on predicted free lengths that approach the ends of their drum windows; the complete formulation is in the repository.

The horizon is $N = 12$ steps of 20 ms. The programme is written in CasADi [37] and solved by IPOPT [38] with the exact Hessian, warm started from the shifted previous solution. After a break declaration the velocity weights are scaled down and the weight of the tension changes up (Table 2), because the elasticity of the cables, which the prediction model leaves out, matters more with seven cables: at the end poses of the recovery plans the lowest elastic mode of the platform falls to 3.2 to 3.6 Hz, against 5.5 Hz at the home pose with eight cables. We do not claim recursive feasibility or closed loop stability for (22). The plant differs from the prediction model through the cable elasticity, the winch dynamics and the estimation errors, and the scenario set changes with time. The tension box itself never becomes infeasible, since it does not depend on the state; what can be lost is the ability to track, and the evaluation measures it instead of assuming it. The bounds apply to the tension commands. The tensions of the elastic cables in the plant follow the commands through the tension loops and can leave the bounds, and the evaluation measures them in the plant, for the intact cables only. When IPOPT returns without success, the controller applies the iterate it returns, with the commands clipped to their bounds; no fallback is used. All 399 unsuccessful solves of the evaluation stopped at the limit of 30 iterations (Section 6.9).

4.2. Accommodation of actuator faults

After the declaration of an actuator fault $\mathscr{A}_i$ the reference is kept, and the tension loop of winch i is corrected by the deficit that the filter of the declared mode estimates, $\hat{t}_i \leftarrow \hat{t}_i - \hat{\delta}_i$ in (17). After a break declaration the winch of the broken cable is held at its angle, so that it does not run away once its load is gone.

4.3. Recovery after a break

Once a break of cable i is declared, the planner decides what the platform can still do, in two branches tried in order. In the first, time scaling, the remaining reference over the next 2 s is slowed down by the largest factor $\kappa \in [0.25, 1]$ for which the margin (14), with the inertial wrench of the slowed motion added to gravity, stays nonnegative at every sample with the seven cables. Below $\kappa = 0.25$ the task no longer continues in any useful sense, and the second branch, recovery, brings the platform to rest within $T = 1$ s at a pose the remaining cables hold with a margin of $s_{\min} = 10$ N. The recovery path solves an optimal control problem on the rigid model with the remaining cables,

$$\begin{aligned}
\min\ & \|\boldsymbol{q}_N - \hat{\boldsymbol{q}}\|^2_{\mathbf{Q}_q} + h\sum_k w_v \|\boldsymbol{v}_k\|^2 + h\sum_k w_a \left\| \frac{\boldsymbol{v}_{k+1} - \boldsymbol{v}_k}{h} \right\|^2 \\
& + w_r \sum_k \|\boldsymbol{t}_{k+1} - \boldsymbol{t}_k\|^2 + w_s \sum_k s_k \\
\text{s.t. } & \boldsymbol{x}_0 = \hat{\boldsymbol{x}}, \quad \text{collocation of (10) on every interval,} \\
& t_{k,i} = 0, \quad t_{\min} + s_p - s_k \leq t_{k,l} \leq t_{\max} - s_p + s_k, \quad l \neq i, \\
& s_k \geq 0, \quad \|\dot{\boldsymbol{p}}_k\| \leq v_{\max}, \quad \|\boldsymbol{\varphi}_k\|_\infty \leq 25^\circ, \quad \boldsymbol{v}_N = \mathbf{0}, \\
& \mathbf{W}(\boldsymbol{q}_N)\boldsymbol{t}_{\text{hold}} + \boldsymbol{w}_g(\boldsymbol{q}_N) = \mathbf{0}, \quad \|\boldsymbol{\varphi}_N\|_\infty \leq 20^\circ, \\
& t_{\min} + s_{\min} \leq t_{\text{hold},l} \leq t_{\max} - s_{\min}, \quad l \neq i,
\end{aligned}$$

(23)

transcribed by Hermite–Simpson collocation [39] on 15 intervals of $h = T/15$ with the tensions constant over each, and solved with IPOPT. The terminal conditions say that the platform ends at rest at a pose the remaining cables hold, and the first term picks among those poses one close to the pose at the break. The twist and acceleration terms keep the motion direct and smooth, and the margin $s_p = 5$ N is relaxed by slacks s_k, which widen the admissible tension region. The dynamics, the attitude, speed and acceleration limits, the drum windows and the rest at the end remain hard constraints, so the slacks do not make the problem feasible from an arbitrary state. A solution with nonzero slacks would mark a third branch, a descent with the least violation of the tension bounds. Three constraints of the implementation are left out of (23) for clarity: a soft tension margin of 20 N, penalised quadratically, which keeps tension in reserve for the feedback of the MPC, the acceleration limit of Table 1, and the drum windows of the remaining cables. We first joined the current state to the nearest safe pose with a quintic polynomial, but after an upper cable break no duration made such a path feasible, because the seven cables can only produce accelerations inside a narrow set that changes with the pose, and a polynomial of fixed shape does not stay in it.

4.4. A library of recovery plans

Solving (23) took 0.21 s in the median and at most 0.82 s on one core, and the platform falls during that time. Because the reference paths of a task are known before the task runs, the planner is therefore evaluated in advance. For each path and each cable, the branch and the plan are computed for break instants every 50 ms from 2.5 to 8.0 s, which covers the possible onsets with room for the detection delay. On a declaration at t_d the controller takes the plan of the instant nearest to $t_d - 15$ ms, the typical delay of a break declaration, and passes it to the MPC as the new reference together with its tensions as feedforward. Every plan after an upper cable break needed the recovery branch, with a median drop of the reference point of 0.54 to 0.60 m depending on the cable, whereas after a lower cable break the time scaling sufficed for 31 to 69 of the 222 instants of both paths and the recovery plans dropped by at most 0.02 m. The platform hanging from seven cables is unstable along these paths: integrated in open loop with the plan's tensions, it drifts from a recovery path after an upper cable break by up to 0.11 m by its end, so the plan has to be tracked in closed loop. All 1,605 recovery problems of the library met the termination criteria of IPOPT, 1,603 at its optimal and 2 at its acceptable level, with every slack zero, so every plan satisfies the tension margins with the tensions it holds over each interval; integrated again interval by interval on the rigid model, the plans deviate from their collocation solution by at most 0.12 mm. This is evidence for the sampled break instants on the rigid model, not a guarantee for other states or for the elastic plant, on which the evaluation tests the plans in closed loop. The library assumes a known task; for other states the planner can still be solved online, and the evaluation includes that variant with its latency.

5. Evaluation

5.1. Simulator

The simulator integrates the full model of Section 2: the rigid platform with its orientation as a unit quaternion, eight elastic unilateral cables (11) and eight winches (12) with the torque and speed limits of their drives. At every millisecond it switches on a fault whose onset has been reached, reads the sensors, hands the controller the reading of the previous sample and integrates the plant over the millisecond in 16 Runge–Kutta steps of 62.5 μs, a step fixed by a convergence test. The plant was verified against a stiff reference solution and against an independent model in MuJoCo [40] (Appendix A). The controller never reads the plant state, and it uses the nominal parameters and the rigid model (10); the gap between plant and prediction model is the one any model based controller meets on a real robot, and keeping it prevents the controller from being tested against its own design model. A run counts as a lost platform, and stops, when one of the three attitude angles (roll, pitch or yaw) exceeds 30°, when P leaves the interior of the frame, or when the drum of an intact cable leaves its window. The rule on the attitude includes yaw, which is not an inclination; Section 6.9 reports which losses it decided on yaw alone and what a rule on roll and pitch alone would change.

Every setting of every method was fixed on calibration runs that share no parameter draw, noise sequence or fault onset with the evaluation, and the protocol of the evaluation, with its methods, runs, endpoints and statistics, was written before any evaluation run. It was amended once: the first evaluation runs showed that the gains of the threshold baseline had been chosen by a rule that ignored the tension bounds, so its settings were chosen again on calibration runs and the evaluation was restarted from its first run, with no setting of any other method changed.

5.2. Methods compared

Table 3 lists the nine methods that we tested and compared. The ablations A1, A2 and A3 remove one part of the proposed controller P each: the probability weighting of the scenarios, the replanning after a break, and the library, whose plan A3 solves online and applies after a latency of 0.4 s, which covers the 99th percentile of the solve times. B0 has no fault tolerance, and the oracle B1 is told the true fault at its onset, which bounds what a perfect diagnosis would achieve with the same controller.

The threshold baseline B2 stands for the classical pipeline: thresholds on the winch quantities for the diagnosis, a computed torque law with the tension distribution (14) instead of the MPC, and the same plan library as P. A baseline is informative only if it is tuned with the same care as the method it is compared with, so its gains

Table 3
Methods of the evaluation. All run on the same plant, sensors and seeds, so every comparison is paired.

Method	Description
P	Proposed: IMM diagnosis; before a declaration the MPC weights the fault scenarios by their probabilities; after a break declaration it follows a plan from the library
A1	P without the probability weighting of the scenarios
A2	P without replanning after a break
A3	P with the recovery plan solved online, applied after 0.4 s
B0	No fault tolerance: healthy model throughout, no declaration
B1	Oracle: the true fault declared at its onset
B2	Thresholds on the winch tension estimates, computed torque control with tension distribution, plans from the library
B3	Recurrent classifier in place of the IMM probabilities for the declaration; A1 otherwise
B3T	B3 with the classifier probabilities also weighting the MPC scenarios

and thresholds were chosen on calibration runs of its own controller. It declares a break when the tension estimate of a winch falls below 9 N. Every threshold on the tension residual that detected an actuator fault also raised false alarms with this controller, so B2 does not isolate actuator faults; it is a break detector, which is what the classical literature on cable robots offers.

The learned baseline B3 replaces the mode probabilities of the IMM by those of a recurrent classifier for the declaration, while the filters of the IMM keep running as the state estimator, so that B3 differs from A1 in the diagnosis alone. In B3T the classifier's probabilities also weight the fault scenarios of the MPC, as the IMM's do in P. The classifier has the same seventeen classes as the IMM and sees the same information: at every estimator cycle, the tension residual $\hat{f}_i - t_{d,i}$, the tension estimate $\hat{f}_i$ and the deviation of the free length from its value at the reference pose, for each winch. A long short term memory network [41] of two layers of 64 units reads the last 0.1 s of these 24 channels; its architecture and hyperparameters came from a search of 30 trials on validation runs [42]. It was trained on 528 runs of both paths, recorded with the controller without the weighting and with declarations disabled, so that the motion carries no trace of any diagnosis. Its outputs were calibrated by temperature scaling [43], and the declaration rule of the IMM was applied to them with thresholds chosen on validation runs.

5.3. Runs, metrics and statistics

Every fault type was injected on every cable of both paths in 13 repetitions: a break, a loss of effectiveness of $\rho = 0.3$ or 0.6, and a torque bias of $\pm 10\%$ of the holding torque, the two values of ρ and the two signs of the bias alternating over the repetitions: $2 \times 3 \times 8 \times 13 = 624$ fault runs, 208 of each type, with the onset drawn uniformly between 3 and 7 s and the run ending 4 s after it or when the platform is lost. There were 100 fault free runs of 16 s per path. Each run draws its plant parameters from its seed (Table 1), and the current noise is nominal in half of the runs and doubled in the other half. All methods ran on the same seeds, so every comparison is paired. Counting every method on every run it applies to, B3T on a subset fixed in advance, the evaluation comprises 5,212 simulations.

Errors are taken to the active reference, the nominal path before a declaration and the plan after it, and the transient is the 2 s after the onset. The diagnosis is scored by the declared mode, its delay and the false alarms, declarations before the onset or in a fault free run counted per hour of operation with the fault modes armed. The recovery time is the time from the onset after which the position error stays for 1 s within twice the 95th percentile of the error in the fault free runs of the same path. Six primary endpoints were fixed with their comparisons, each with a Holm correction [44]: the platform lost after a break, the fault declared with the correct mode, the delay of the correct declarations, the false alarms per hour, the smallest tension margin in the transient and the share of the transient outside the bounds. Shares carry 95% Wilson intervals [45] and paired comparisons McNemar's exact test [46]; continuous endpoints are compared with the Wilcoxon signed rank test [47] and bootstrap intervals [48]. No run was excluded.

6. Results

Table 4 gives the outcome of every method; the complete statistics of the six endpoints are in the repository given under Data availability.

6.1. One break in detail

Fig. 4 follows one break, chosen by a rule fixed before drawing: among the upper cable breaks whose traces were kept, the run with the median declaration delay of P. Cable 2 breaks at 4.933 s on T1. The IMM gives the break mode a probability above 0.99 in the cycle 11 ms after the break and declares it after 17 ms. The classifier's probability rises earlier, to 0.99 at 6 ms, but both reach the threshold of 0.9999 in the same cycle, 11 ms after the break; the IMM needs two such cycles and declares after 17 ms, the classifier needs four and declares after 27 ms, and the threshold rule of B2 declares at 22 ms. The platform cannot stay where it is. Under P it descends by 0.5 m along the recovery plan to a pose the seven remaining cables can hold, tilting by at most 6.7° and settling at 5.3°, while the seven intact cables take over the load of cable 2: an intact cable leaves its bounds for 5% of the first 2 s, all of it within 0.21 s of the break, and from 0.3 s on every intact tension lies between 15 and 183 N. Without fault tolerance the platform turns past 30° and is lost 0.40 s after the break.

6.2. Detection and isolation

The IMM declared 623 of the 624 faults with the correct mode (Fig. 5). Its median delay was 17 ms for a break,

Table 4
Outcome of every method. Lost: platforms lost in the runs with a break of an upper and of a lower cable, out of the break runs of each group; correct: faults declared with the correct mode, out of all fault runs of the method; delay: median delay of the correct break declarations; false alarms: per hour of operation with the fault modes armed; recovered: break runs after which the position error returned to the band of the fault free runs. A2 and A3 ran the 208 break runs only, and B3T a subset fixed in advance of 192 fault runs, 64 of them breaks, and 60 fault free runs. B3T lost 17 of its 44 platforms in break runs before the break, every other method after it. The differences in platforms lost between P and each of B0, B2, B3 and A2 remain after the Holm correction ($p \leq 0.007$).

	Lost		Correct	Delay	False alarms	Recovered
Method	upper	lower		(ms)	(per h)	breaks
P	11/104	0/104	623/624	17	0.8	181/208
A1	11/104	0/104	624/624	17	0.0	182/208
A2	31/104	7/104	208/208	17	–	74/208
A3	20/104	0/104	208/208	17	–	173/208
B0	104/104	18/104	–	–	–	0/208
B1	3/104	1/104	624/624	–	–	197/208
B2	35/104	30/104	208/624	22	3.1	93/208
B3	24/104	1/104	598/624	30	32.5	157/208
B3T	28/32	16/32	16/192	532	505.1	19/64

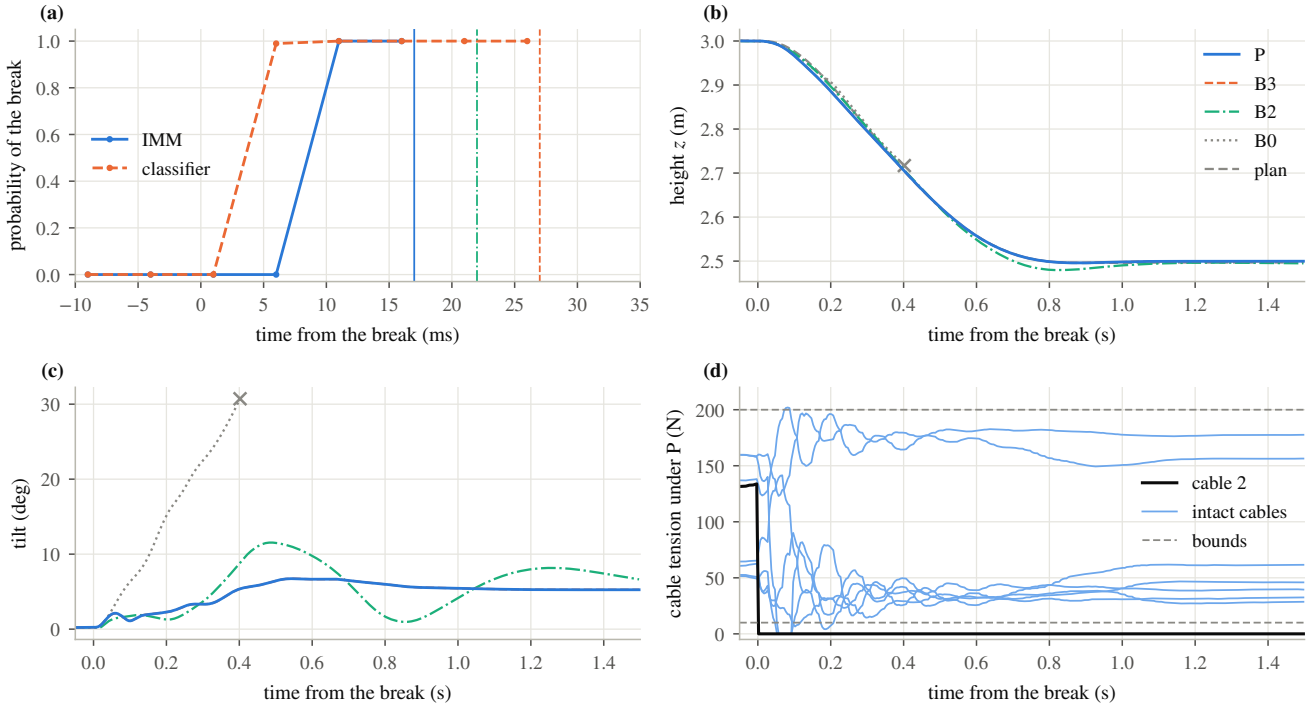


Figure 4: A break of the upper cable 2 at 4.933 s on T1 (run 1 of the evaluation), drawn from the stored traces. (a) Probability of the true mode from the IMM and from the classifier up to their declarations; vertical lines: declarations of P (17 ms), B2 (22 ms) and B3 (27 ms). (b) Height of the platform and the recovery plan followed by P. (c) Tilt, the angle of the platform axis from the vertical; the cross marks the loss of the platform without fault tolerance (B0), when its roll passed 30°. (d) Tensions of the elastic cables in the plant under P, with the bounds that the MPC imposes on the tension commands: the seven intact cables take over the load of the broken one. The curves of B3 lie under those of P.

23 ms for a loss of effectiveness and 41 ms for a bias, and no break took longer than 19 ms. Its one false alarm, 2.8 s before a loss of effectiveness of the same winch, is also the one fault it did not declare correctly. On the same runs P declared 5 ms earlier than B2 and 10 ms earlier than B3 (paired medians). B2 declared every break correctly and no actuator fault, and in 21 runs with a loss of effectiveness of an upper winch it declared a break of a healthy lower cable whose tension had fallen below its threshold. The classifier of B3 declared all 208 breaks correctly, 206 of the 208 losses of effectiveness and 184 of the 208 biases.

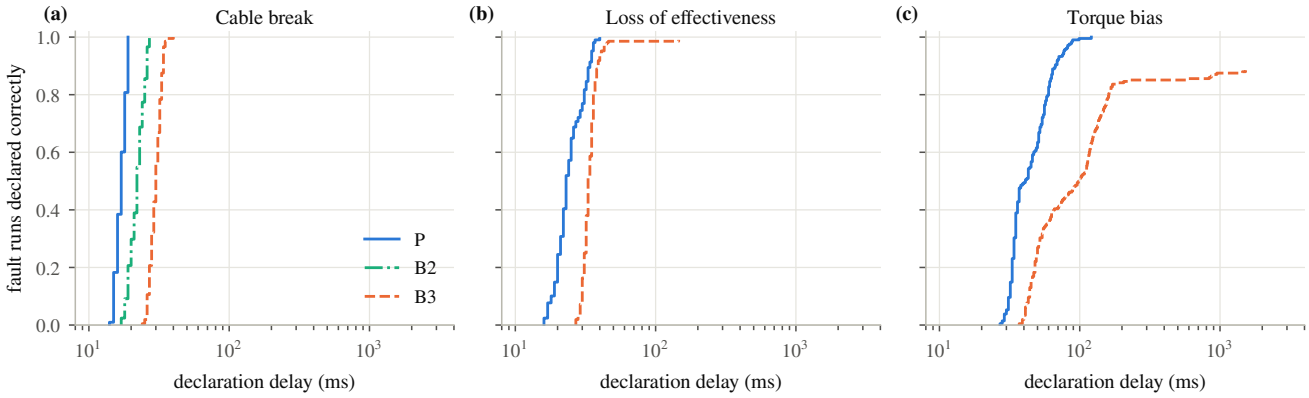


Figure 5: Share of the 208 fault runs of each type (both paths) declared with the correct mode within a given delay after the onset. The IMM (P) declares every break within 19 ms and isolates losses of effectiveness and biases faster and more completely than the classifier (B3); the threshold rule (B2) isolates breaks only. A curve ends at the share of correct declarations.

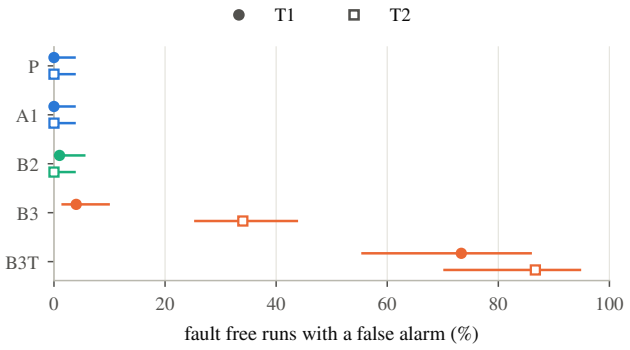


Figure 6: Share of the fault free runs with a false alarm, with 95% Wilson intervals (100 runs of 16 s per path; 30 per path for B3T). The IMM raised none with or without the weighting; the classifier raised many on the helix T2, and many more when its probabilities also weighted the MPC (B3T).

The difference that matters most in closed loop is the false alarm. The classifier raised 42 false alarms in the 1.29 h of operation in which P raised one, and its fault free runs carried a false alarm in 4 of 100 on T1 and in 34 of 100 on T2 (Fig. 6). Training on both paths brought its isolation close to that of the IMM, but not its rate of false alarms on the helix (Fig. 7). A first classifier trained on T1 alone had shown how strongly the learned diagnosis depends on its training data: on the helix it raised a false alarm in 6 of 8 fault free test runs and declared 45 of 80 faults correctly, taking the healthy motion along the climbing path for actuator faults of the lower winches, while the IMM, which uses a model of the robot rather than examples of its motion, kept its performance on both paths.

6.3. Safety after a break

The proposed controller lost the platform after 11 of the 208 breaks (5.3%), against 122 without fault tolerance (58.7%), 65 for B2 (31.2%), 25 for B3 (12.0%) and 38 without replanning (18.3%); every one of these differences remains after the Holm correction. All 11 losses of P followed the break of an upper cable, 6 of 52 on T1 and 5 of 52 on T2, and P kept the platform after every lower cable break (Fig. 8). The oracle B1 lost 4, three of them after upper cable breaks, so even a diagnosis without delay cannot save every upper cable break with this controller. The price of the 17 ms of diagnosis appears in the tension bounds rather than in the losses: during the 2 s after a break a cable was outside its bounds for 5.1% of the time under P and 2.1% under the oracle (medians). B2, with its gains tuned on its own calibration runs and the same plans, lost 65 platforms, 30 of them after a lower cable break, after which P never lost one.

6.4. Recovery

Replanning carries most of this result. Without it (A2), 38 platforms were lost, 86.5% of the transient was spent outside the bounds in the median, and the error after the transient was 72.7 mm against 5.6 mm for P. The library matters as well: solving the plan online with its latency (A3) lost 20 platforms against 11 (paired, $p = 0.09$, a secondary comparison) and raised the time outside the bounds in the transient from 5.1 to 20.2%. With P the position error returned to the band of the fault free runs in 181 of the 208 break runs, after 0.27 s in the median (Fig. 9); the oracle recovered 197 runs after 0.19 s, A3 173 after 0.70 s and A2 74 after 1.01 s.

6.5. Actuator faults

No method that isolates actuator faults lost a platform after one, except B3T, which lost 14 after the onset of an actuator fault and 37 more in such runs before the onset (Section 6.6). B2, which cannot isolate them, lost 18 platforms after a loss of effectiveness, and in three other runs with an actuator fault it lost the platform before the onset, after a false alarm. The accommodation of P is biased, however. After a loss of effectiveness, the IMM's estimate of the deficit had an RMS error of 30 N in the median, 20 N for

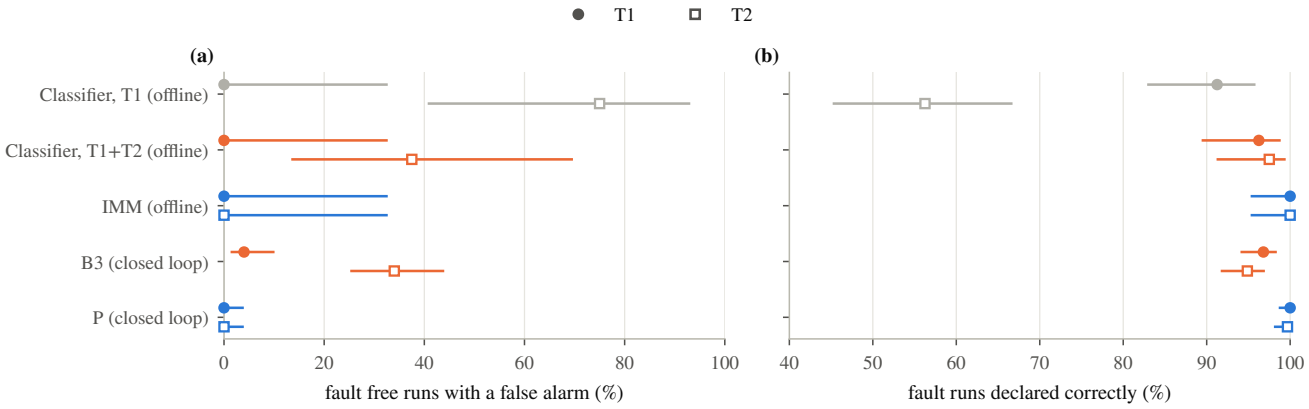


Figure 7: Learned against model based diagnosis, offline and in closed loop, with 95% Wilson intervals. (a) Fault free runs with a false alarm. (b) Fault runs declared with the correct mode. Offline: single tests on the same stored runs (8 fault free and 80 fault runs per path) of the classifier trained on T1, of the classifier trained on both paths and of the IMM. Closed loop: the evaluation (100 fault free and 312 fault runs per path), in which the classifier (B3) replaces the mode probabilities of the IMM for the declaration while the filters of the IMM remain the state estimator. Training on both paths removes most of the failure to generalise to the helix, but the false alarms on T2 remain.

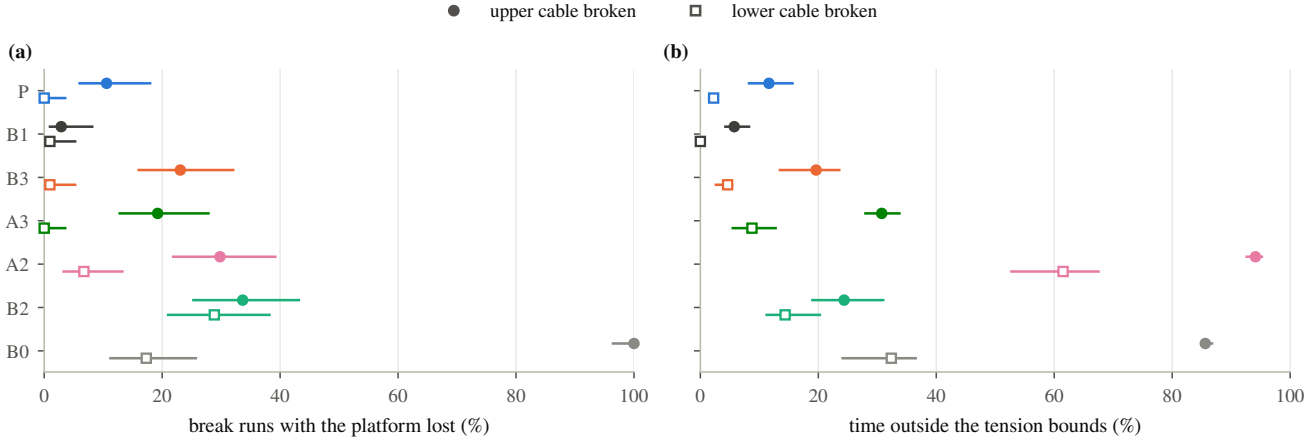


Figure 8: Safety after the 104 upper and 104 lower cable breaks of the evaluation. (a) Share of the runs in which the platform was lost, with 95% Wilson intervals. (b) Median share of the 2 s after the break during which an intact cable was outside its tension bounds, with 95% bootstrap intervals. All losses of P follow upper cable breaks; without replanning (A2) the platform spends most of the transient outside the bounds.

$\rho = 0.3$ and 52 N for $\rho = 0.6$, and the error after the transient was 18.2 mm, 14 and 45 mm for the two magnitudes, so that only 80 of the 208 runs returned to the band of the fault free runs. After a bias the deficit error was 5.6 N and 206 of 208 runs returned to the band. The cause is the structure of the accommodation: the IMM keeps the tension commands as its input while the loop of the faulty winch is corrected by the deficit it estimates, so its filter then estimates the deficit that remains after the correction, and the pair settles at about half of the true deficit.

6.6. The probability weighting of the scenarios

With the IMM the weighting had no measurable effect on the safety endpoints (Fig. 10). P and A1 lost the same number of platforms after a break, 11 each, with the outcome differing in ten runs, five each way, and their tension margins in the transient did not differ (paired median difference 0 N, $p = 0.26$). Their declarations differed in two respects, which a comparison of the paired runs after the evaluation showed and which the analysis plan did not include. P raised one false alarm, 2.8 s before a loss of effectiveness that A1 declared correctly, and in 51 of the 208 runs with a torque bias the two declared at different times, P earlier in 49 of them, by 5 to 15 ms, and later in two; breaks and losses of effectiveness were declared at the same times. Before a fault the weighting was almost never active: the position error before the fault was identical in 804 of the 824 paired runs. The reason is the speed with which breaks are declared. Every break was declared within

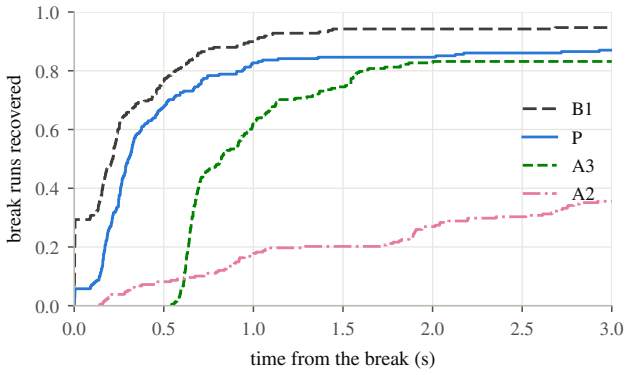


Figure 9: Share of the 208 break runs whose position error has returned within the given time to twice the 95th percentile of the fault free error of the same path, and stayed there for 1 s. Lost runs and runs that did not recover before their end never reach the curve. The library (P) recovers almost as fast as the oracle (B1); solving the plan online (A3) costs about 0.4 s, and without replanning (A2) most runs do not recover.

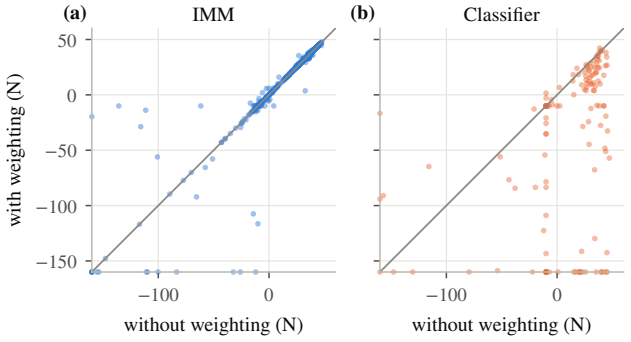


Figure 10: Smallest tension margin of the intact cables in the 2 s after the fault, with the weighting against without it on the same run. (a) IMM (P against A1, 624 runs): the points lie on the diagonal. (b) Classifier (B3T against B3, 138 runs): most points lie below it. Values below −160 N are drawn at −160 N.

19 ms, less than the 20 ms period of the MPC, so the MPC solves at most once with intermediate probabilities before it switches to the model of the declared mode. The weighting can only act while the diagnosis is uncertain, and for a break that interval is shorter than one control period.

With the classifier the closed loop became much worse. On the runs of B3T, with its probabilities weighting the scenarios of the MPC, a false alarm was raised in 154 of 252 runs against 11 for B3 on the same runs, the correct declarations fell from 186 to 16 of 192, the platform was lost in 44 of 64 break runs against 8, in 17 of them before the cable broke, and the margin in the transient fell by 19 N (paired median; 95% interval −28.7 to −13.8 N). Unsuccessful solves of the MPC came with this. IPOPT returned without success in 382 solves in 111 of the 252 runs of B3T, against one solve in one run for P. The replay of Section 6.9 recorded every solve of the lost runs: all unsuccessful ones stopped at the limit of 30 iterations, 318 of the 335 with the healthy and two fault scenarios in the programme, and the iterates they returned violated the equality constraints of the prediction by 0.14 in the median and by up to 116 (in metres, radians or their rates), against at most 0.004 for the eight unsuccessful solves in the lost runs of A2. Of the 114 platforms B3T lost, 73 were lost before any fault, 19 of them in fault free runs; 61 of these 73 runs contained an unsuccessful solve, 22 a false declaration, and 51 no declaration at all. In each of the 78 lost runs with an unsuccessful solve the first one came before the loss, by 0.53 s in the median, and in 55 of them the attitude was still below 10° when it came; in 33 other runs the solver failed and the platform was kept. The unsuccessful solves are therefore a plausible part of the mechanism, but these runs cannot separate how much the probabilities of the classifier, the motion the weighting produced and the convergence of the solver each contributed. The first pilot run of B3T showed one path: as soon as the classifier gave the fault modes some probability, their scenarios entered the MPC, the motion this produced resembled nothing in the classifier's training runs, which were recorded without the weighting, and the classifier then declared an actuator fault on a healthy winch.

6.7. Computation

The simulation waits for each solve, so the closed loop behaved as if every computation met its period. On the machine of the evaluation, with two runs sharing two cores, they did not all do so (Fig. 11). The estimator cycle of the observers and the IMM took 2.9 ms on average and 5.3 ms at the 99th percentile, against its period of 5 ms. The MPC took 32 ms on average and 65 ms at the 99th percentile, against its period of 20 ms, so the controller as implemented, with CasADi and IPOPT and the models compiled to C, would not run in real time on this machine, and Section 6.8 shows that it needs its solutions on time.

6.8. Sensitivity to the computation time of the MPC

In the evaluation every output of the MPC acted in the sample in which its solve began, although the solves took 32 ms on average (Section 6.7). After the evaluation we tested what this assumption carries, under a protocol written and published before the runs. P ran again on the 104 upper cable breaks, which contain all its losses, with the output of the MPC available a fixed computation time D after its solve begins. For $D > 0$ a solve begins every $\max(20\ \text{ms}, D)$ and uses the state estimate, the reference and the command applied at that instant; its first tension command and the feedforward of the drum acceleration take effect D later and are held until the next solution takes effect. The observers, the IMM, the declaration and the tension loops keep their timing, and neither the controller nor its settings were changed after the results were seen. With $D = 0$ all 104 rows were identical to those of the evaluation.

The controller did not tolerate any of the delays (Table 5). With the output one period late, $D = 20$ ms, the

Table 5
Sensitivity of P to the computation time D of the MPC on the 104 upper cable breaks of the evaluation, a study added after the evaluation, with its protocol fixed before its runs. For $D > 0$ a solve starts every $\max(20\ \text{ms}, D)$, and its first command takes effect D after the start and is held until the next. Lost: platforms lost, with the 95% Wilson interval in percent; before: of these, lost before the cable broke; p: exact McNemar test against $D = 0$ on the same runs, Holm correction over the three delays; outside and margin: medians over the runs that reached the break, of the share of the 2 s after it with an intact cable outside its bounds and of the smallest tension margin of the intact cables (–: no run reached the break); correct: breaks declared with the correct mode.

D (ms)	Period (ms)	Lost	95% CI (%)	Before	p	Outside (%)	Margin (N)	Correct
0	20	11/104	6–18	0	–	11.6	-10	104/104
20	20	104/104	96–100	104	< 0.001	–	–	0/104
40	40	104/104	96–100	104	< 0.001	–	–	0/104
60	60	104/104	96–100	104	< 0.001	–	–	0/104

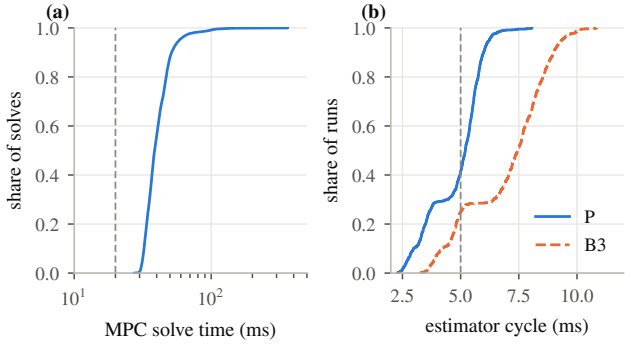


Figure 11: Computation on the machine of the evaluation (two runs sharing two cores). (a) Every MPC solve of P in the kept fault runs against the 20 ms period. (b) 99th percentile of the estimator cycle of every run of P and of B3 (with the classifier) against the 5 ms period. The simulation waited for every computation, so these times did not delay any command in the evaluation; Section 6.8 delays the output of the MPC.

platform was lost in all 104 runs, and every loss came before the cable broke, between 0.66 and 1.40 s after the start, while the breaks were due between 3 and 7 s. The roll of the platform oscillated at about 5 Hz with an amplitude that grew by a factor of 32 in the median over half a second, and every run ended on roll. In every run the IMM, not yet armed, then declared a fault of an intact cable or winch, a break of an upper cable in 100 of them, and the platform was lost 0.17 to 0.55 s later. The declaration did not start the motion, since in 77 runs roll or pitch had already passed 5°, but it may have hastened the loss; the runs do not separate the two. With 40 and 60 ms the platform was lost in every run as well, again before the break, and unsuccessful solves appeared in 68 and 55 of the runs. The largest computation time tested without significantly more losses than at $D = 0$ is therefore 0 ms.

The MPC, with weights fixed on calibration runs in which each solution acted at once, thus needs its solution within a small part of its period, and the results of this paper hold for a controller that meets this, not for the implementation timed in Section 6.7. A fixed delay is a model of the computation time and not a real time implementation: the study shows the sensitivity to the delays tested, not whether a given computer meets them. Delays shorter than one period, and an MPC that predicts the state at the instant its command takes effect, were not tested.

6.9. Replay of the lost runs

After the evaluation we simulated again every one of the 432 runs in which a platform was lost, with the code and the settings of the evaluation and with two additions to the record: the attitude angle that ended the run and the outcome of every solve of the MPC. All 432 rows came out identical to the stored ones. This shows that the stored results follow from the code; it does not show that the rule of Section 5.1 is the right physical criterion for a lost platform. Roll ended 398 of the runs, pitch 26 and yaw 8. In one of the eight, pitch had passed 30° as well; in the other seven, runs of A2 and B2, yaw alone had. We continued these seven runs without the rule on the attitude. In six of them roll or pitch passed 30° within 0.46 s of the stored end, and so did the tilt of the platform axis; in the seventh, a break of an upper cable under B2, roll, pitch and tilt stayed below 30° until the run ended. A rule on roll and pitch alone, which can only end fewer runs than the rule we used, therefore changes one outcome. B2 then loses 64 platforms after a break instead of 65, and its difference from P remains after the Holm correction ($p = 4.0 \times 10^{-11}$). No other loss changes, and every loss of P ended on roll. We also replayed the 41 runs in which a solve was unsuccessful but the platform was kept, again with identical rows. All 399 unsuccessful solves of the evaluation, the number the campaign had stored, stopped at the limit of 30 iterations. B3T had 382 of them; P had one, 1.2 s after the break of an upper cable, in a run in which the platform was kept.

7. Discussion

7.1. Why the model based diagnosis was faster and more reliable

The IMM separates faults through what each mode predicts, and the predictions differ where the physics of Section 3.1 says they should. A break makes the tension

estimate of one winch fall to zero while its command does not, which only the break mode of that cable explains, and this is why every break was declared within four estimator cycles. An actuator fault leaves the winch estimate at its command and moves the platform, which the length readings of all eight cables register, so the actuator modes need more evidence and take longer. The filters also take the controller's own tension commands as their input, and so they carry a model of the motion the controller produces. The classifier has no such model. It learned the motion of its training runs, and when the motion differed, on the helix it had not seen or under the weighting, it read the difference as a fault. Training on more varied runs narrows this gap, but in a fault tolerant controller the controller itself creates new motion once it acts on the detector, and that motion cannot be in the training data of a detector trained before the controller used it. Used for the declaration alone, the classifier cost 14 platforms more than the IMM, 25 against 11; used also to weight the scenarios of the MPC, it lost the platform in 44 of 64 break runs, together with many unsuccessful solves of the MPC (Section 6.6). We present B3 as one carefully built learned diagnosis that shares the state estimator of the IMM, and not as the best a learned detector can do: the result belongs to this model, its training runs and this closed loop, and a detector trained on more runs, more paths and runs that include the controller's reaction would do better.

7.2. What the library buys and what it assumes

The plans of the library are the same plans that the online planner computes; the difference between P and A3 is only when they are available. With the 0.4 s of latency, nine more platforms were lost, a difference that is suggestive but not significant on its own ($p = 0.09$), while the share of the transient outside the bounds quadrupled and the recovery took 0.70 s instead of 0.27 s in the median. Together they indicate that after an upper cable break the robot has a fraction of a second, not seconds, to start moving. The price of the library is that it needs the task in advance. For a robot that repeats known paths, as on a production line or a building site where the trajectories are planned before the work, this is a small price, and for other states the online planner remains, with its latency. The remaining losses of P all follow upper cable breaks, and even the oracle lost four platforms. Every plan of the library satisfies the tension margins on the rigid model, so these losses arise when the closed loop fails to follow a plan that exists, either because the state at the declaration has already drifted from the start of the plan or because the platform on seven elastic cables, with its lowest mode near 3 Hz, does not respond as the rigid model of the MPC predicts. Which of the two dominates we have not analysed run by run.

7.3. Why the weighting had little to act on

The weighting of (22) assumes an interval in which the diagnosis suspects a fault without having confirmed it, and with the IMM that interval was shorter than one period of the MPC for a break; for a torque bias, declared after 41 ms in the median, it was long enough to move some declarations earlier, without a measurable effect on safety. It would act with a slower diagnosis, or more precisely with a diagnosis that stays uncertain for several control periods, but with the only slower diagnosis we tested, the classifier, the closed loop became worse. Its probabilities were not those of a model of the closed loop, they brought fault scenarios into the MPC before any fault, and the solver then often stopped at its iteration limit; which of these did the damage, the runs cannot tell. Whether a diagnosis can be slow enough to leave the controller time to hedge and still be calibrated to the motion the hedging produces is a question this study leaves open.

7.4. Limitations

The controller does not run in real time, and its results depend on its solutions arriving on time: with the output of the MPC delayed by one period the platform fell in every run tested (Section 6.8). The MPC needed 32 ms on average against its period of 20 ms, because it solves a nonlinear programme with the exact Hessian to convergence at every step. A real time iteration scheme [49], which performs one sequential quadratic programming iteration per sample, implemented in a solver built for embedded optimal control such as acados [50], or a shorter horizon, are the next steps, possibly with a prediction of the state at the instant the command takes effect; each of them changes the controller, which then has to be evaluated again.

The accommodation of actuator faults is biased (Section 6.5). Giving the IMM the corrected command removes the bias in the steady state, and in pilot runs it cut the error after a loss of effectiveness of 0.3 from 25 to 2 mm, but it lost the platform for $\rho = 0.6$, where the present scheme kept it. An accommodation that estimates the deficit from the corrected command therefore needs a stability argument which the present design does not have.

The study is a simulation. The geometry and inertia of the robot are those of a published model, but the cable stiffness is scaled from a larger cable, the motor is sized from the manufacturer's data, the noise of the current measurement is our choice, and friction is not modelled. Friction in the gearbox and at the drum would enter the tension estimate (16) like a torque bias, as an error that changes sign with the winch speed, and would have to be identified and compensated on hardware. The evaluation randomises the parameters and doubles the noise to cover part of this uncertainty, but a test on a physical robot is the step that remains. It also covers single faults only: the IMM and the declaration logic handle one fault and do not rearm after a declaration, so a second fault during the recovery is outside its scope.

8. Conclusions

A cable break on this robot is a race against the fall of the platform, and the race is won by isolating the fault quickly and having the recovery motion ready. A two level diagnosis, observers at the winches and an IMM over

seventeen modes at the platform, isolated every fault type from the encoders and motor currents alone, declared every break within 19 ms and raised one false alarm in 1.29 h. With recovery plans computed in advance along the task the platform was lost after 11 of 208 breaks, all of them upper cable breaks, against 122 without fault tolerance and 20 when the same plans were solved online. A learned classifier trained on the same signals was slower and less reliable, and when its probabilities also weighted the scenarios of the MPC the closed loop raised false alarms in most runs and the solver often stopped without converging. Weighting fault scenarios by the probabilities of the IMM, the idea the study started from, had no effect because the IMM declares a break within one control period. These results assume that each solution of the MPC acts within its sample; delayed by one period, the controller lost the platform before any fault in every run of the added study. The work therefore stops short of a real time implementation and of an unbiased accommodation of actuator faults, and its next steps are a real time iteration of the MPC, an accommodation with a stability argument, double faults and the test on a laboratory robot that would show whether the margins found in simulation hold on hardware.

CRediT authorship contribution statement

Mfeuter Joseph Tachia: Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft. **Alexander V. Maloletov:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was financially supported by the Laboratory of the Institute of Artificial Intelligence, Innopolis University.

Data availability

The code of the robot model, the simulator, the estimators, the controller and the learned classifier, the configuration files, the stored results of the evaluation with the complete statistics of its endpoints, and the scripts that rerun every step are available at `https://github.com/Tachia/cdpr-fault-tolerant`, the code under the MIT licence and the documents and data under CC BY 4.0.

Declaration of generative AI use

During the preparation of this work the authors used Claude (Anthropic), a large language model used through the Claude application with code execution, to write and run the simulation, analysis and plotting code to the authors' design, to draft the manuscript text from the authors' study design and results, and to check the bibliographic records of the references. The figures were rendered by these plotting scripts from the stored simulation results; no figure was generated or altered by the tool itself. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Acknowledgements

The authors thank the Laboratory of the Institute of Artificial Intelligence, Innopolis University, for its financial support of this work.

A. Verification of the model and the simulator

Table 6 lists the checks of the model of Section 2 and of the simulator. All of them run automatically on the code that produced the results.

Table 6
Verification of the model and the simulator. The model checks use 100 random poses within ± 0.5 m and ± 0.5 rad of the home pose; the simulator checks use the parameters of the study.

Check	Criterion	Result
$\partial \boldsymbol{l}/\partial \boldsymbol{q} = -\mathbf{W}^\top \mathbf{H}$ against central differences	relative error $< 10^{-7}$	9.0×10^{-10}
$\mathbf{M} = \mathbf{H}^\top \mathbf{M}_v \mathbf{H}$ against a symbolic Lagrangian	absolute error	1.8×10^{-15}
$\boldsymbol{G} = -\mathbf{H}^\top \boldsymbol{w}_g$ against the symbolic gradient	absolute error	7.1×10^{-15}
$\dot{\mathbf{M}} - 2\mathbf{C}$ skew symmetric, $\dot{\mathbf{M}}$ exact	$< 10^{-10}$	7.1×10^{-15}
Lagrange form (8) against Newton–Euler form (10)	largest difference of $\dot{\boldsymbol{\nu}}$	9.9×10^{-11}
Energy with locked winches and undamped cables, 10 s at 62.5 μs	drift $< 10^{-6}$ of the initial kinetic energy	3.5×10^{-11}
Static equilibrium at the home pose	tensions of (14)	151.4 and 58.6 N; largest $\lvert\dot{\boldsymbol{x}}\rvert = 4.8 \times 10^{-10}$
Step halving from 62.5 to 31.25 μs, five fault scenarios	metrics change $< 0.1\%$	all but one (0.12%, the peak tension of an upset platform)
Reference solution, Radau IIA [51, 52], lower cable break		5.8×10^{-8} m, 0.010 N
Independent model in MuJoCo, same inputs, slack cables, 5 s	difference $< 1\%$ of the motion	1.4×10^{-10} of the motion

References

[1] A. Pott, Cable-Driven Parallel Robots: Theory and Application, Springer Tracts in Advanced Robotics, Springer, Cham, 2018. doi:10.1007/978-3-319-76138-1.

[2] S. Qian, B. Zi, W.-W. Shang, Q.-S. Xu, A review on cable-driven parallel robots, Chinese Journal of Mechanical Engineering 31 (2018) 66. doi:10.1186/s10033-018-0267-9.

[3] M. J. Tachia, A. V. Maloletov, Applications of dynamic models for cable-driven parallel robots: a comprehensive review, Russian Journal of Nonlinear Dynamics 22 (2026) 139–200. doi:10.20537/nd251101.

[4] J.-B. Izard, A. Dubor, P.-E. Hervé, E. Cabay, D. Culla, M. Rodriguez, M. Barrado, Large-scale 3D printing with cable-driven parallel robots, Construction Robotics 1 (2017) 69–76. doi:10.1007/s41693-017-0008-0.

[5] A. Pott, H. Mütherich, W. Kraus, V. Schmidt, P. Miermeister, A. Verl, IPAnema: A family of cable-driven parallel robots for industrial applications, in: A. Pott, T. Bruckmann (Eds.), Cable-Driven Parallel Robots, volume 12 of *Mechanisms and Machine Science*, Springer, 2013, pp. 119–134. doi:10.1007/978-3-642-31988-4_8.

[6] R. G. Roberts, T. Graham, T. Lippitt, On the inverse kinematics, statics, and fault tolerance of cable-suspended robots, Journal of Robotic Systems 15 (1998) 581–597. doi:10.1002/(SICI)1097-4563(199810)15:10<581::AID-ROB4>3.0.CO;2-P.

[7] L. Notash, Failure recovery for wrench capability of wire-actuated parallel manipulators, Robotica 30 (2012) 941–950. doi:10.1017/S0263574711001160.

[8] A. Berti, M. Gouttefarde, M. Carricato, Dynamic recovery of cable-suspended parallel robots after a cable failure, in: Advances in Robot Kinematics 2016, Springer Proceedings in Advanced Robotics, Springer, Cham, 2018, pp. 331–339. doi:10.1007/978-3-319-56802-7_35.

[9] C. Passarini, D. Zanotto, G. Boschetti, Dynamic trajectory planning for failure recovery in cable-suspended camera systems, Journal of Mechanisms and Robotics 11 (2019) 021001. doi:10.1115/1.4041942.

[10] G. Boschetti, G. Carbone, C. Passarini, Cable failure operation strategy for a rehabilitation cable-driven robot, Robotics 8 (2019) 17. doi:10.3390/robotics8010017.

[11] R. Boumann, T. Bruckmann, Development of emergency strategies for cable-driven parallel robots after a cable break, in: Cable-Driven Parallel Robots (CableCon 2019), volume 74 of *Mechanisms and Machine Science*, Springer, 2019, pp. 269–280. doi:10.1007/978-3-030-20751-9_23.

[12] R. Boumann, C. Jeziorek, T. Bruckmann, Validation of emergency strategies for cable-driven parallel robots after a cable failure, in: Cable-Driven Parallel Robots (CableCon 2023), Mechanisms and Machine Science, Springer, Cham, 2023, pp. 209–220. doi:10.1007/978-3-031-32322-5_17.

[13] V. Di Paola, S. Caro, M. Zoppi, Design and performance investigation of a sliding-mode adaptive proportional–integral–derivative control for cable-breakage scenario, Meccanica 59 (2024) 1927–1937. doi:10.1007/s11012-024-01875-2.

[14] H. A. P. Blom, Y. Bar-Shalom, The interacting multiple model algorithm for systems with Markovian switching coefficients, IEEE Transactions on Automatic Control 33 (1988) 780–783. doi:10.1109/9.1299.

[15] Y. Zhang, X. R. Li, Detection and diagnosis of sensor and actuator failures using IMM estimator, IEEE Transactions on Aerospace and Electronic Systems 34 (1998) 1293–1313. doi:10.1109/7.722715.

[16] Y. Zhang, J. Jiang, Bibliographical review on reconfigurable fault-tolerant control systems, Annual Reviews in Control 32 (2008) 229–252. doi:10.1016/j.arcontrol.2008.03.008.

[17] A. Raman, I. Walker, V. Krovi, M. Schmid, Cable failure tolerant control and planning in a planar reconfigurable cable driven parallel robot, Frontiers in Robotics and AI 10 (2023) 1070627. doi:10.3389/frobt.2023.1070627.

[18] Y. Majdi, H. Marufkhani, M. A. Khosravi, Active sliding mode fault tolerant control of a fully-constrained planar cable-driven parallel robot, in: Proc. 11th RSI International Conference on Robotics and Mechatronics (ICRoM), 2023, pp. 762–768. doi:10.1109/ICRoM60803.2023.10412615.

[19] S. M. Fazeli, M. Abedi, A. Molaei, M. Hassani, M. A. Khosravi, A. Ameri, Active fault-tolerant control of cable-driven parallel robots, Nonlinear Dynamics 111 (2023) 6335–6347. doi:10.1007/s11071-022-08184-x.

[20] J. Bettega, G. Piva, D. Richiedei, A. Trevisani, Load torque estimation for cable failure detection in cable-driven parallel robots: a machine learning approach, Multibody System Dynamics 64 (2025) 485–513. doi:10.1007/s11044-024-10023-3.

[21] E. Khalastchi, M. Kalech, On fault detection and diagnosis in robotic systems, ACM Computing Surveys 51 (2018) 1–24. doi:10.1145/3146389.

[22] M. Gouttefarde, J. Lamaury, C. Reichert, T. Bruckmann, A versatile tension distribution algorithm for n-DOF parallel robots driven by $n+2$ cables, IEEE Transactions on Robotics 31 (2015) 1444–1457.

doi:10.1109/TRO.2015.2495005.

[23] J. C. Santos, M. Gouttefarde, A. Chemori, A nonlinear model predictive control for the position tracking of cable-driven parallel robots, IEEE Transactions on Robotics 38 (2022) 2597–2616. doi:10.1109/TRO.2022.3152705.

[24] C. Song, D. Lau, Workspace-based model predictive control for cable-driven robots, IEEE Transactions on Robotics 38 (2022) 2577–2596. doi:10.1109/TRO.2021.3139585.

[25] X. Gao, W. Shang, B. Zhang, Disturbance observer-based model predictive control for cable-driven parallel robots, IEEE Transactions on Robotics 41 (2025) 6307–6326. doi:10.1109/TRO.2025.3626664.

[26] S. Lucia, T. Finkler, S. Engell, Multi-stage nonlinear model predictive control applied to a semi-batch polymerization reactor under uncertainty, Journal of Process Control 23 (2013) 1306–1319. doi:10.1016/j.jprocont.2013.08.008.

[27] D. Lau, J. Eden, Y. Tan, D. Oetomo, CASPR: A comprehensive cable-robot analysis and simulation platform for the research of cable-driven parallel robots, in: Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), 2016, pp. 3004–3011. doi:10.1109/IROS.2016.7759465.

[28] P. Tempel, P. Miermeister, A. Pott, Kinematics and dynamics modeling for real-time simulation of the cable-driven parallel robot IPAnema 3, in: Proc. 14th IFToMM World Congress, Taipei, Taiwan, 2015. doi:10.6567/IFToMM.14TH.WC.OS4.020.

[29] Bosch Rexroth AG, Synchronous Servomotors MSK: Project Planning Manual, 12 ed., Bosch Rexroth AG, 2018. DOK-MOTOR*-MSK********-PR12-EN-P, R911296289; MSK030C data in Table 4-5. The values used are identical in edition 05 (2006), titled Rexroth IndraDyn S MSK Synchronous Motors.

[30] K. H. Hunt, F. R. E. Crossley, Coefficient of restitution interpreted as damping in vibroimpact, Journal of Applied Mechanics 42 (1975) 440–445. doi:10.1115/1.3423596.

[31] H. J. Ernst, Der E-Modul von Seilen unter Berücksichtigung des Durchhanges, Der Bauingenieur 40 (1965) 52–55.

[32] M. J. Tachia, A. V. Maloletov, Reinforcement learning-based control for cable sag compensation in cable-driven parallel robots using soft actor-critic algorithm, in: Proc. 2025 VI International Conference on Control in Technical Systems (CTS), 2025, pp. 171–176. doi:10.1109/CTS67336.2025.11196696.

[33] E. Idà, V. Mattioni, Cable-driven parallel robot actuators: State of the art and novel servo-winch concept, Actuators 11 (2022) 290. doi:10.3390/act11100290.

[34] J. Han, From PID to active disturbance rejection control, IEEE Transactions on Industrial Electronics 56 (2009) 900–906. doi:10.1109/TIE.2008.2011621.

[35] Z. Gao, Scaling and bandwidth-parameterization based controller tuning, in: Proc. 2003 American Control Conference, volume 6, 2003, pp. 4989–4996. doi:10.1109/ACC.2003.1242516.

[36] B.-Z. Guo, Z.-L. Zhao, On the convergence of an extended state observer for nonlinear systems with uncertainty, Systems & Control Letters 60 (2011) 420–430. doi:10.1016/j.sysconle.2011.03.008.

[37] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, M. Diehl, CasADi: a software framework for nonlinear optimization and optimal control, Mathematical Programming Computation 11 (2019) 1–36. doi:10.1007/s12532-018-0139-4.

[38] A. Wächter, L. T. Biegler, On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming, Mathematical Programming 106 (2006) 25–57. doi:10.1007/s10107-004-0559-y.

[39] M. Kelly, An introduction to trajectory optimization: How to do your own direct collocation, SIAM Review 59 (2017) 849–904. doi:10.1137/16M1062569.

[40] E. Todorov, T. Erez, Y. Tassa, MuJoCo: A physics engine for model-based control, in: Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), 2012, pp. 5026–5033. doi:10.1109/IROS.2012.6386109.

[41] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (1997) 1735–1780. doi:10.1162/neco.1997.9.8.1735.

[42] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: Proc. 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2019, pp. 2623–2631. doi:10.1145/3292500.3330701.

[43] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proc. 34th International Conference on Machine Learning (ICML), volume 70 of *Proceedings of Machine Learning Research*, 2017, pp. 1321–1330.

[44] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (1979) 65–70.

[45] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (1927) 209–212. doi:10.1080/01621459.1927.10502953.

[46] Q. McNemar, Note on the sampling error of the difference between correlated proportions or percentages, Psychometrika 12 (1947) 153–157. doi:10.1007/BF02295996.

[47] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (1945) 80–83. doi:10.2307/3001968.

[48] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993.

[49] M. Diehl, H. G. Bock, J. P. Schlöder, A real-time iteration scheme for nonlinear optimization in optimal feedback control, SIAM Journal on Control and Optimization 43 (2005) 1714–1736. doi:10.1137/S0363012902400713.

[50] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, M. Diehl, acados—a modular open-source framework for fast embedded optimal control, Mathematical Programming Computation 14 (2022) 147–183. doi:10.1007/s12532-021-00208-8.

[51] E. Hairer, G. Wanner, Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems, 2 ed., Springer, Berlin, 1996. doi:10.1007/978-3-642-05221-7.

[52] P. Virtanen, et al., SciPy 1.0: fundamental algorithms for scientific computing in Python, Nature Methods 17 (2020) 261–272. doi:10.1038/s41592-019-0686-2.